Memory for VLA Policies: from EventVLA, TRACE and SAI to a 103-paper landscape — the awesome_memory_vla consolidated report

Three deep dives · ten insights · design-space matrix · annotated paper list

2026-09-07

> This report is assembled by `scripts/build_pdfs.py` from the trends report, the three deep dives, the design-space matrix and the 103 paper notes in the repository. The authoritative version of each chapter is the corresponding Markdown file. Every number is taken from the original paper or abstract; numbers from different papers rest on different task suites and scoring rules and cannot be compared directly.

Contents

---

1. Trends and Insights

> This is the synthesis part of awesome_memory_vla. The three core deep dives are EventVLA, TRACE and SAI; all 103 entries are listed in the README. Every number below is taken from the original paper or abstract. Numbers from different papers rest on different task suites and scoring rules and cannot be compared directly.

1. What is happening in the field

1.1 Timeline: the June–August 2026 burst

By submission month, the 97 non-background papers in this repository split as: 24 in all of 2025, 23 in January–May 2026, then **20 in June, 10 in July and 17 in August 2026**. All three core papers appeared in mid-June (TRACE June 12, SAI June 15, EventVLA June 18), in the same month as KEMO (June 22), μVLA

(June 10), MemoryWAM (June 18), WeaveLA (June 16), PRISM/ReMemBench (June 15) and the long-context diffusion-policy study (June 15). This is not coincidence: RMBench (March 1), RoboMME (March 4) and RoboMemArena (May 11) landed in spring and gave method papers a shared target, and the summer papers are the first round of answers.

1.2 A shared starting point

Nearly every first paragraph says the same thing: VLAs are Markovian (`a_t = π(o_t, l)`) and manipulation is not. The papers differ in their diagnosis of *why not*, and the diagnosis determines the memory design:

Diagnosis	Typical phrasing	Representative work	Design tendency
Evidence vanishes	Lift a cover, glimpse, close it; the cue leaves the view	EventVLA, TRACE, KEMO, UniMem	Sparse, explicit, addressable
State aliasing	Visually identical states at different stages (2nd vs 3rd button press)	KEMO, TFP, ChainVLA, WeaveLA, CAMP	Task-progress belief, event counting
Spatial forgetting	A single wrist camera loses objects that leave the view	AtlasVLA, AnchorVLA4D, SAM2Act+	Persistent spatial state, initial anchor frame
Partner unobservable	The partner's phase can only be inferred from history	SAI, Duet	History token + training-distribution coverage
No progress tracking	A frozen VLA executes chunks open-loop without knowing where it is	AGM, HELM, BATON, HyMeS	External symbolic state + verifier

1.3 Five technical families

The 97 papers fall into five families by *where memory lives and in what form* (README sections 3–7):

1. **Event / keyframe memory** (EventVLA, KEMO, Keyframe-Chaining, UniMem, WeaveLA, Bi-HIL, MemoryWAM's anchor layer, MemER's high-level frame selection): write only when something happened, store raw frames or their tokens.
2. **Dense compressed history** (MemoryVLA / MemoryVLA++, ContextVLA, CronusVLA, HAMLET, NativeMEM, StreamPI, TempoFit, DySta, FibVLA, PRISM, HALO, LaMem-VLA, Remember Smarter): every frame enters; cost is controlled by compression, tokenisation, KV reuse or sampling.
3. **Latent state / recurrent / slot memory** (TRACE, μVLA, VPWEM, VQ-Memory, MemoAct, RB-VLA, MTIL, DSSP, Chronos, TFP, CAMP, GMP, ELMUR, AEM, FM-VLA): history compressed into fixed-size vectors or slots updated recursively.
4. **Dual-system / agentic / symbolic memory** (MEM, MemER, MAP-VLA, Notes-to-Self, Explicit Language Memory, CodeGraphVLP, HyMeS, HELM, Harness VLA, AGM, PonderPounce, BATON, OnEvoMemory, ChainVLA): a VLM / LLM / code layer maintains text, graphs or progress pointers; a low-level VLA executes.

5. **Memory inside world models** (MemoryWAM, MemoryVAM, TriVLA, the imagination branch of MemoryVLA++): memory serves prediction, prediction serves action.

A sixth direction, **partner memory in multi-robot systems** (SAI, HATS, Duet, Tri-Manual, AutoIntervene), has only a handful of papers but shares the same mathematics with the other five.

2. The design space: six questions every memory VLA must answer

Question	EventVLA	TRACE	SAI	Other typical answers
What is stored (content)	Raw image frames	512-D latent of vision + state	GRU hidden state	One token per frame (NativeMEM), K/V (TempoFit), text (MEM, Notes-to-Self), semantic graph (CodeGraphVLP), force series (FM-VLA), voxel state (AtlasVLA)
When to write (trigger)	Learned future-keyframe probability + NMS + cooldown	Every step; gates decide how much	Every step	Kinematic + visual rules (KEMO), event classifier (UniMem), subgoal completion (WeaveLA), advance only after physical verification (AGM), value-guided (OnEvoMemory)
Where to write / read (addressing)	Temporal concatenation, attention finds it	Path signature of the robot trajectory	None (single vector)	Content similarity (MemoryVLA, HALO), progress-aware query (Keyframe-Chaining), trajectory-similarity retrieval of demos (MAP-VLA), LRU (ELMUR)
How much (capacity)	5 frames + 3–4 anchors	K = 4/6 slots	30 steps	Fixed token count (VPWEM, μVLA's m tokens), linear growth (frame stacking, StreamPI), unbounded text (MEM)
Where it plugs in (integration point)	Input to the VLM vision encoder	Adapter before the action head	Decoder input	Action-expert conditioning (TFP, WeaveLA, FM-VLA), KV cache (TempoFit), prompt (MAP-VLA, PonderPounce)
What supervises it	Qwen3-VL keyframe labels + soft targets + curriculum	No labels; three stabilisers	Data curriculum + interventions	Time-contrastive (HAMLET), past-token prediction (PTP), VQA distillation (HALO), TBPTT (μVLA), imitation loss only (most)

How to read the table: the two extremes were claimed in the same month by two papers. EventVLA puts its weight on *when to write*; TRACE on *where to read*. The middle ground, learning both the write trigger and the addressing, is still empty.

3. Evidence across benchmarks

Publicly reported numbers on the three 2026 benchmarks (each paper's own protocol):

RMBench (9 tasks, RoboTwin 2.0)

Method	Avg.	Note
$\pi_{0.5}$ (no memory)	10.4–11.2	As reported by EventVLA and Chronos
MemoryVLA (QwenOFT)	41.7	EventVLA's reproduction

Method	Avg.	Note
Mem-0 (RMBench's modular policy)	42.0	
ChainVLA	62.8	1.2B; 11.2 without Motion Tail, 3.0 without Progress Context
EventVLA (anchors only)	67.8	KEM not enabled
Chronos	73.6	10× fewer parameters than $\pi_{0.5}$, 30× fewer than Mem-0

RoboMME (16 tasks; temporal / spatial / object / procedural memory; $\pi_{0.5}$ backbone)

Method	Number
$\pi_{0.5}$ current-observation	17.93
FrameSamp+Modul (RoboMME's strongest variant)	44.51 (57.88 with 9× data)
PonderPounce 0.8B / 9B	50.04 / 60.83 (75.54 with 9× data)
WeaveLA	Hardest repetition slice SwingXtimes N=3: 0% → 47.8%; single-execution episodes unchanged
AGM	Beats the strongest memory baseline on the Counting subset (PickXTimes, BinFill)

RoboMemArena (26 tasks, > 1000 steps on average, 68.9% of subtasks memory-dependent)

Method	Cumulative / task success
$\pi_{0.5}$	52.5 / 41.3
PrediMem (the benchmark's dual-system policy)	4.5 / 14.5 below HyMeS
HyMeS ("skills in weights, memory in code")	66.2 / 60.1
BATON	+14.9 / +11.6 over SoTA

MIKASA-Robo (32 tasks, RL lineage)

Method	Number
MemoryVLA	41.2 (+11.8 over CogACT / π_0)
VPWEM	> 20% over diffusion policies and VLAs
μVLA (minimal recurrence)	Five training tasks 0.42 → 0.84; held-out 0.07 → 0.23
ELMUR	Best on 21 of 23 tasks; aggregate about 70% above previous best

Real-robot delayed / transient evidence

Paper	Number
EventVLA (ARX ACONE, 4 tasks × 20)	90 / 60 / 90 / 75 vs π_MEM 50 / – / 30 / 40

Paper	Number
TRACE (5 tasks × 25)	ACT 25.50 → 69.23, DP 25.00 → 59.53, $\pi_{0.5}$ 50.47
KEMO (dual-arm, 830–2846 steps)	Task success +23.6, stage completion +34.1 over $\pi_{0.5}$
NativeMEM	Simulation 32.4 → 84.0, real robot up to 98.7, competitive with 20% of the data
UniMem	Simulation 93.4 vs 68.2 for fixed-interval sampling; hardware 80.0 vs 43.5 for hierarchical baseline
Chronos (single RGB, dual-arm)	78% over 4 tasks, 72% on the memory-dependent subset; $\pi_{0.5}$ 7% / 0%

Three observations. First, on RMBench "initial frame plus recent window" already reaches 67.8%, so that benchmark's memory demand is shallow, and Chronos's 73.6% comes from a full-history SSM rather than explicit event memory. Second, RoboMME and RoboMemArena are led by dual-system / agentic methods (PonderPounce, HyMeS, BATON): when the task needs **counting and procedural memory**, symbolic progress state is currently more reliable than end-to-end latents. Third, on real transient-evidence tasks the highest absolute success comes from end-to-end event memory (EventVLA, KEMO, UniMem). **No single memory leads on every benchmark**; RoboMME's conclusion that representation effectiveness is highly task-dependent has been confirmed by a full year of results.

4. Ten insights

Insight 1: the frontier moved from "how much to store" to "when to write"

The 2025 memory VLAs competed on window length and compression (CronusVLA, ContextVLA, MemoryVLA). The mid-2026 papers turn almost simultaneously to **write timing**: EventVLA learns future-keyframe probabilities, KEMO detects events from kinematics plus visual change, UniMem trains an event classifier, WeaveLA fires on subgoal completion, MemoryWAM uses event-boundary anchor frames, TFP's mechanistic analysis finds write gain about 6× larger near manipulation events than in non-event phases, OnEvoMemory learns what to keep from rollout outcomes. Different architectures, one conclusion: **most of history is redundant with the present; the frames worth writing are the moments of state transition.**

Insight 2: "Present but Not Remembered" supplies the mechanism

Liao & Cao audit three frozen VLAs with layer-resolved linear probes and causal interchange interventions: past-frame content is linearly decodable throughout the network, but **information unique to the history, absent from the current frame, is nearly missing**; stored history is largely a copy of the present, and it is used causally only when the current frame is heavily degraded. This explains why frame stacking gives inconsistent gains (ContextVLA's starting point) and why sparse event frames work: an event frame carries exactly what the current frame lacks. It yields a design rule for memory augmentation: **inject information unique to the past, not more history.** TRACE's signature increment `δ_t`, CAMP's compressed action history and PTP's past-token prediction all read as instances of this rule.

Insight 3: raw frames or latents depends on how many bits must be recovered

EventVLA's implicit-bank ablation (24.9% vs 75.2%) and TRACE's success with 512-D slots (69.23) look contradictory; they measure two kinds of information. EventVLA's tasks require remembering the colours, positions and order of 3–5 objects at once: high entropy, spatial detail. TRACE's tasks require "which side" or "which book": one or two bits of branch variable. RoboMME's finding (representation effect depends on the task) becomes operational: **estimate how much information the downstream decision must recover from the past, then choose the representation.** Detail-heavy recall wants raw frames or dense tokens (EventVLA, NativeMEM, UniMem's dense spatial memory); branch variables want slots or beliefs (TRACE, TFP, RB-VLA); counting wants symbols (AGM's progress pointer, Notes-to-Self's scratchpad).

Insight 4: addressing is the under-rated axis

Most work defaults to "concatenate in time and let attention find it". TRACE is one of few papers that treats addressing as a first-class problem: trajectory signatures as keys, route similarity around 90 under order-preserving transforms and 37.8 under reversal. Keyframe-Chaining's progress-aware query, MAP-VLA's trajectory-similarity retrieval and ELMUR's LRU slots are three other addressing schemes. The value of addressing is **a stable correspondence between write time and read time**: TRACE uses the path travelled, progress-aware methods use the stage reached. When a task's branch structure does not correspond to robot motion (the cue is only a colour, motions are identical), trajectory addressing degrades; that boundary is the next thing to solve.

Insight 5: long context is less brittle than claimed, but spurious correlation is real

Agarwal et al. (long-context diffusion policies) sweep context length systematically and conclude that naive scaling is not as brittle as the literature says: with UNet + cross-attention and ordinary data volumes, single-task policies reach high success on many tasks. The opposing evidence is equally solid: PTP shows diffusion policies *ignore* past–future action dependencies (the inverse of copycat), GMP finds that simply extending history drops performance through distribution shift and overfitting (gated memory +30.1 over long-history baselines on MemMimic), HALO needs VLM-prior distillation and sparse attention to suppress spurious correlations. The reconciliation: long context works when supervision steers attention toward task-relevant history. EventVLA's soft labels and curriculum, TRACE's balance / entropy / consistency losses and μVLA's TBPTT are all doing this job.

Insight 6: reliability comes from disciplined state updates, not capacity

AGM's thesis: if external memory treats "attempted" as "completed", a local execution error becomes a persistent task-state error, so the progress pointer advances only after physical evidence verifies the subgoal. The same discipline appears as EventVLA's NMS + cooldown (one event cannot flood the buffer), TRACE's gated writes (unrouted slots barely move), BATON's verifier agent (the VLA is invoked only after the wrist view confirms readiness) and HELM's state verifier (predict failure before execution). **A memory**

write should be a verified transaction: a principle discovered independently on both the end-to-end and the agentic path.

Insight 7: dual systems still lead on counting and procedural memory

The leaders on RoboMME and RoboMemArena (PonderPounce, HyMeS, BATON) all have a high level that manages state and a low-level VLA that executes. HyMeS states it most bluntly: skills in weights, memory in code; motor skills come from imitation, memory-management heuristics from a coding agent iterating on rollout feedback. EventVLA's critique of dual systems (latency, error propagation) holds on transient-visual-evidence tasks, but for discrete procedural memory ("press three times", "skip the finished subtask") symbolic state is currently steadier. PonderPounce uses an MLLM's native causal context as memory and asynchronously passes only the newest cognition token (p50 78 ms refresh, 25 ms action), narrowing the latency gap. **The end-to-end versus dual-system line is shifting from "which is better" to "which information belongs at which level".**

Insight 8: the cost of memory is heading toward zero

EventVLA's multi-frame input cuts throughput from 2.91 Hz to 0.94 Hz, the most expensive in this batch; contemporaneous work heads the other way. TRACE adds 2.1 ms per step; NativeMEM reuses the VLA's own vision encoder to compress each frame per view into one token; TempoFit trains nothing and adds no tokens, reusing prefix K/V at intermediate layers; StreamPI adds zero parameters, using instruction-anchored causal attention and length extrapolation for streaming multi-frame inference; DySta keeps one copy of static tokens and reuses their KV cache, 2× faster with higher success. **"Nearly free memory" is now an attainable engineering target**, and EventVLA's threefold latency will be the first thing its successors optimise.

Insight 9: multi-robot collaboration is the same problem wearing another face

SAI treats the partly visible partner as a data-curriculum problem, yet its own ablation shows the history token is decisive for execution reliability (premature release 64.5% → 16.1%). Treat partner phase as a latent inferred from history and SAI's setting is isomorphic to TRACE's delayed evidence, with "early cue" replaced by "what the partner just did". Current multi-robot work is almost entirely about data collection (HATS: one human plus an agent for four arms; Duet: human–human demonstrations for pretraining; Tri-Manual: offline re-timing), and **nobody has yet used an explicit memory module for partner-state estimation**. That is a clear gap.

Insight 10: evaluation is fragmenting while diagnostics mature

Within a year at least twelve memory-related benchmarks appeared: RMBench, RoboMME, RoboMemArena, RoboTwin-MeM, ReMemBench, LIBERO-Mem, MemMimic, MemoryRTBench, RuleSafe, Memory-T-Bench, MemoryBench, LongBench, with different task sets and scoring, so cross-paper

numbers cannot be compared (Section 3 could only tabulate per benchmark). Meanwhile **diagnostics** are maturing: TRACE's route similarity / branch consistency with an order-reversal negative control, Present-but-Not-Remembered's probes and causal interventions, TFP's hidden-state interventions, LongBench's split of long-horizon failure into execution robustness and context dependence. The next round of evaluation should report both "success" and "what the memory actually used".

5. Open problems

1. **Buffer saturation.** EventVLA's five-frame FIFO evicts early evidence on > 10-minute event-dense tasks; TRACE's fixed slots drop to 0.928 consistency at 2× history. Hierarchies (recent / event / gist, as in MemoryWAM) are the current compromise and have not been compared head-to-head with flat memories on one benchmark.
2. **Distinguishable addresses.** Trajectory signatures need branches to correspond to different motions; their dimension grows cubically with state dimension (17-D → 5219; depth 4 → 88,740). High-DoF humanoids and hands need log-signatures or learned low-dimensional trajectory keys.
3. **Supervision for the write trigger.** EventVLA labels offline with a 235B model, KEMO uses rules, UniMem a classifier, OnEvoMemory rollout outcomes. Which generalises across tasks has not been tested in a controlled way.
4. **Coupling between memory and chunk length.** EventVLA's foresight collapses when the chunk shrinks from 50 to 15 (75.2 → 13.6); WhyChunking identifies non-Markovian expressivity as one of chunking's benefits. Memory mechanisms and chunk length should be designed jointly.
5. **Safety of wrong memories.** AGM shows unverified memory freezes local errors into state; EventVLA does not discuss recovery from a wrongly committed frame.
6. **Explicit memory for multi-robot systems.** See Insight 9.
7. **Benchmark unification.** RoboMME's four memory types (temporal / spatial / object / procedural), RoboTwin-MeM's n-parameterisation and TRACE's stage-progress metric could be assembled into one reporting template.

6. What the three papers teach about method

- **Isolate a default and test it alone.** EventVLA changes only the memory representation (raw vs latent); TRACE changes only the addressing (with / without signature routing, with / without increments); SAI changes only the data curriculum (architecture fixed). The persuasiveness of all three comes from single-variable controls.
- **Report the cost.** EventVLA reports latency (0.94 Hz), TRACE reports milliseconds per step and parameter increments, SAI reports intervention data as a share of the baseline (30%). Readers can then judge whether the gain is worth it.
- **Diagnose instead of guessing.** TRACE's order-reversal negative control is the single most reusable design here: it turns "the memory uses history" from an assumption into a measurement.

7. Where we would go next (research directions)

1. **Foresight writes × trajectory addressing.** Use EventVLA's KEM head as the write trigger and TRACE's signatures as the slot address, and test whether performance holds on both RoboTwin-MeM (high-entropy visual evidence) and TRACE's tasks (low-entropy branch variables). Hypothesis: sparse writes solve capacity, trajectory addresses solve read/write alignment.
2. **Partner-phase memory.** In SAI's two-robot setting, replace A's 30-step GRU with a TRACE-style signature-routed slot memory (key = A's own trajectory, content = observations of B) and test whether Yield/Wait stays stable as B's delay grows.
3. **A memory-diagnostics benchmark.** Attach order-preserving and order-breaking history transforms to each of RoboMME's four task types and make route similarity / branch consistency a standard report item for every memory VLA.
4. **Cross-task transfer of write triggers.** On one backbone, compare EventVLA (learned), KEMO (rule-based) and UniMem (classifier) write policies on unseen tasks, to answer whether write timing is task-specific or general.

---

2. EventVLA Deep Dive

> **Paper**: EventVLA: Event-Driven Visual Evidence Memory for Long-Horizon Vision-Language-Action Policies
> **Authors**: Ganlin Yang, Zhangzheng Tu, Yuqiang Yang, Sitong Mao, Junyi Dong, Tianxing Chen, Jiaqi Peng, Jing Xiong, Jiafei Cao, Jifeng Dai, Wengang Zhou, Yao Mu, Tai Wang (USTC / Shanghai AI Lab / Huawei / HKU / Tsinghua / PKU / SJTU)
> **arXiv**: 2606.20092 (2026-06-18) · Code and data: InternRobotics/EventVLA
> **In this repo**: English PDF · Chinese PDF · 中文解读

0. One sentence

EventVLA lets the VLA policy predict which of the next 50 steps will turn out to be worth remembering, stores only those raw frames in a five-slot buffer, and feeds them back into the vision encoder together with the first frame and a short recent window. On the authors' new RoboTwin-MeM benchmark this lifts average success from 18.0% (anchors only) to 75.2%; on RMBench the anchors alone reach 67.8% against 42.0% for the previous best (Mem-0); on four real bimanual tasks it scores 90 / 60 / 90 / 75%.

1. The problem

A standard VLA is Markovian, `a_t = π(o_t, l)`, which quietly assumes everything relevant stays visible. Real manipulation violates this constantly: the robot lifts a cover, glimpses a colour, closes the cover; reads a random number card; watches a stick point out an order. The evidence appears for a moment and is needed hundreds of steps later. The paper calls these non-Markovian manipulation tasks and sorts existing memory-augmented approaches into three families, each with a concrete failure mode:

Family	Examples	Failure mode
Dual system (high-level VLM manages memory, low-level policy acts)	MemER, Mem-0, MEM (π_MEM)	High latency, errors propagate across the hierarchy
Recurrent / hidden-state compression	AVA-VLA, Recurrent Memory Transformer	Information bottleneck discards fine visual detail
Frame buffers (keep past raw frames)	MemoryVLA, CronusVLA, ContextVLA, LoLA	Unselective accumulation drowns sparse evidence in redundancy and costs compute

The question the paper sets itself: **exactly when, and what, should a VLA preserve so that it succeeds without overwhelming its compute budget?**

2. Method

2.1 Memory structure: anchors plus event frames

The policy becomes `a_t = π(o_t, M_{t−1}, l)` with `M_t = A_t ∪ E_t`:

- **Foundational visual anchors** `A_t = {o_0} ∪ {o_{t−K}, …, o_{t−1}}`. The initial frame fixes the invariant global layout; the short window supplies motion and progress cues. Rule-based, nothing to learn. Simulation uses `o_0, o_{t−30}, o_{t−15}`; the real robot uses `o_0, o_{t−60}, o_{t−40}, o_{t−20}`.
- **Event keyframes** `E_t`, written by the KEM module, capped at `N_max = 5`, FIFO eviction.

At inference the three parts are concatenated in temporal order, `I_input = concat([A_t, E_{t−1}, o_t])`, and pushed straight through the VLM vision encoder. There is no separate read module: reading is ordinary multi-frame self-attention.

2.2 KEM: predicting future keyframe probabilities

KEM is a light MLP head next to the action head. Its input is the final-layer hidden state `h_t ∈ R^{H×d}` of the autoregressive transformer (`H = 50`, the action chunk); its output is one probability per future step:

```
p̂_t = σ(KEM_mlp(h_t)) = [p̂_t^1, …, p̂_t^H] ∈ [0,1]^H
```

`p̂_t^i` is the probability that step `t+i` is a task-critical keyframe. Predicting over the whole chunk rather than step by step matters because events often appear and vanish midway through a chunk, where a per-step classifier would be too late. Sharing `h_t` is the key design decision: the hidden state already encodes observation and action queries, so the head knows what the policy is about to do and can "schedule" a write in advance.

Write rule: when `p̂_t^i ≥ τ_commit` (0.55), the raw image at `t+i` is committed to `E_t`. Two post-processing steps keep one event from flooding the buffer with near-duplicate frames:

1. **1-D NMS** keeps only local maxima within a window of radius `w = 8`;

2. **Cooldown** drops any candidate less than `C = 10` steps after the last committed frame.

2.3 Training: automatic labels, soft targets, teacher-to-student curriculum

- **Labels** come from Qwen3-VL-235B-A22B (8× A800, vLLM) watching demonstrations offline: 128 uniformly sampled multi-view frames per episode, few-shot prompted to emit keyframe steps as JSON. Against physics-engine ground truth in simulation the mean temporal error is under 10 steps; against human labels on the real robot it stays within 50 steps (episodes of 1500–2000 steps).
- **Soft targets**. Hard 0/1 labels destabilise the head because physical events are temporally fuzzy; the labels are smoothed with a raised-cosine kernel of radius `R`: `y_t^i = 0.5(1 + cos(π|t+i−t*|/R))`.
- **Loss**: `L = L_action + λ L_kem`, with `L_kem` a sequence-averaged BCE and `λ = 0.1`.
- **Curriculum**. Early in training the buffer is built from ground-truth keyframes (teacher forcing) with probability `α`, which decays linearly from 1 to 0 so the policy ends up relying on its own thresholded predictions. This closes the train/test gap.

2.4 Backbone and hyper-parameters

Setting	RMBench / RoboTwin-MeM	Real robot
Base	QwenOFT (Qwen3-VL-4B-Instruct, StarVLA codebase)	$\pi_{0.5}$ (PaliGemma)
Action head	OFT, `H = 50`, 14-D actions	flow head, `H = 50`, 32-D actions
Training	80k steps, VLM lr 1e-5, new modules 1e-4	60k steps, 5e-5 cosine, global batch 32
Images	224×224	224×224
KEM	τ=0.55, N_max=5, w=8, C=10, λ=0.1	same

3. The RoboTwin-MeM benchmark

The authors argue that RMBench and RoboMME can mostly be solved from the first frame plus a recent window, so evidence that appears only transiently mid-interaction has never been isolated. RoboTwin-MeM (RoboTwin 2.0, SAPIEN) has eight bimanual tasks, each tagged with `n ∈ [1, 5]`, the number of intermediate keyframes that must be retained:

Task	n	Avg. steps	What is tested
Rearrange Blocks Hard	1	879	Which block was moved
Put Back Block Hard	2	1468	Which outer pad each block visited
Pick Objects in Order / Pick the Unhidden Block	3	1124 / 699	Lift covers, close them, pick by order / by elimination
Cover Blocks Hard / Find Seal Stamp / Reproduce Route	4	1544 / 1338 / 1417	Remember colours across covers; return the seal under its cover; copy a demonstrated route (in-context)

Task	n	Avg. steps	What is tested
Press Button Keyframe	2–5	430	Read two number cards, press buttons that many times (counting)

Three capabilities are separated: transient evidence memory (cover tasks), sequence and counting (buttons), and in-context imitation (Reproduce Route). Each task has 50 demonstrations with per-step language annotations.

4. Results

4.1 RMBench: anchors are enough

Method	Avg. success
$\pi_{0.5}$ / X-VLA / QwenOFT (no memory)	10.4 / 9.8 / 5.6
MemER / Mem-0 (dual system)	8.7 / 42.0
MemoryVLA (OpenVLA) / MemoryVLA (QwenOFT)	19.4 / 41.7
EventVLA without initial frame / without short-term window	33.7 / 23.8
EventVLA (visual anchors only)	**67.8**

RMBench's nine tasks depend on persistent layouts and fixed motion styles, so only the anchor version was deployed. Both anchor components are necessary: removing either drops the score to 33.7% or 23.8%. Per task, Rearrange Blocks 96%, Put Back Block 95%, Swap Blocks 96%, Cover Blocks 97%, but Observe and Pick Up 21% and Press Button 3%: counting is beyond anchors.

4.2 RoboTwin-MeM: KEM is the qualitative jump

Method	n=1	n=2	n=3	n=3	n=4	n=4	n=4	n=5	Avg.
$\pi_{0.5}$	20	19	1	14	0	8	0	0	7.8
MemER	32	4	12	2	0	26	3	5	10.5
MemoryVLA (QwenOFT)	39	0	1	9	1	11	0	25	10.8
EventVLA anchors only	62	13	5	20	0	26	0	18	18.0
EventVLA anchors + KEM	**62**	**93**	**90**	**54**	**94**	**63**	**98**	**48**	**75.2**

(Columns: Rearrange Blocks Hard, Put Back Block Hard, Pick Objects in Order, Pick the Unhidden Block, Cover Blocks Hard, Find Seal Stamp, Reproduce Route, Press Button Keyframe.)

The entire jump from 18.0% to 75.2% comes from KEM. At n=1 anchors already suffice (62% unchanged); for n≥2 almost all the success comes from event frames. Mem-0 scores 0% on this suite.

4.3 Ablations: what each design choice is worth

Variant	Avg.	Reading
Full EventVLA	75.2	—
Implicit memory bank (keyframes compressed into one latent)	24.9	Several events squeezed into one vector: information bottleneck
Hard labels	48.8	Unstable head, missed writes
No NMS	53.4	Adjacent frames of one event flood the buffer; early evidence evicted
N_max = 2	32.0	Not enough capacity
Chunk 30 / 15	31.1 / 13.6	Foresight window too short to schedule writes

“Implicit bank 24.9 vs raw frames 75.2” is the most consequential number in the paper: it refutes the default assumption that compressing history into a latent is enough, at least for tasks that need three to five distinct visual details at once.

4.4 No regression on Markovian tasks

On standard RoboTwin 2.0 tasks EventVLA improves on its QwenOFT base, Easy 80.0→83.8% and Hard 78.0→81.6%, and edges $\pi_{0.5}$ (82.7 / 76.8).

4.5 Real robot: ARX ACONE bimanual

Four tasks, 20 trials each: Find Block Easy / Hard (the hidden block’s location is only briefly visible), Pick-X-Times (read a random number, count), Pick in Order (a stick points out a sequence).

Method	Find Block Easy	Find Block Hard	Pick-X-Times	Pick in Order
$\pi_{0.5}$	0–10%	0–10%	0–10%	0–10%
π_MEM (reproduced)	50	—	30	40
EventVLA	**90**	**60**	**90**	**75**

4.6 The cost: throughput drops to a third

Method	Latency (s/chunk)	Throughput (chunks/s)
QwenOFT	0.36	2.91
EventVLA anchors only	0.96	1.07
EventVLA anchors + KEM	1.09	0.94

Multi-frame input multiplies the visual tokens; throughput falls from 2.91 Hz to 0.94 Hz. The authors’ defence is that VLAs act as high-level planners over high-frequency low-level controllers, so 1 Hz is workable.

5. Assessment

5.1 The real contributions

1. **“When to write” becomes a prediction instead of a rule.** Previous methods wrote every frame, sampled at fixed intervals, or delegated the decision to an external VLM. KEM predicts write probabilities for the next 50 steps from the policy’s own hidden state, so the policy effectively reserves a memory slot for something it is about to see. It shares the representation with action prediction and adds almost no parameters.
2. **A clean controlled comparison of raw frames versus latents.** 75.2 versus 24.9 was measured inside one framework, on one dataset, changing only the memory representation. That is far more credible than cross-paper comparison.
3. **Memory demand parameterised by n.** RoboTwin-MeM makes “how many things must be remembered” an explicit task attribute, so success can be plotted against n. Earlier memory benchmarks could not offer that diagnostic.

5.2 Points to keep in mind

1. **KEM was never evaluated on RMBench.** The 67.8% there is the anchor-only model; KEM’s gains appear only on the authors’ own RoboTwin-MeM. Both benchmarks come from the RoboTwin ecosystem (RMBench’s first author Tianxing Chen is also an author here), whose design naturally favours the anchors-plus-keyframes hypothesis. No RoboMME or MIKASA-Robo numbers are reported.
2. **KEM is tightly coupled to long action chunks.** Shrinking the chunk from 50 to 15 collapses success from 75.2% to 13.6%, below the anchors-only model (18.0%). Foresight is a function of chunk length, so the mechanism will not transfer directly to tasks that need short chunks and fast reaction (contact-rich or dynamic scenes).
3. **A capacity-5 FIFO ceiling.** The limitations section concedes that tasks over 10 minutes with dense events will saturate the buffer and evict early evidence. Even within the benchmark, the hardest n=5 task scores 48% and Find Seal Stamp (n=4) 63%: success clearly declines with n.
4. **Labelling depends on a 235B model.** Running Qwen3-VL-235B on eight A800s for offline annotation is a cost the paper does not quantify. Real-robot keyframe error of up to 50 steps is absorbed by the soft labels.
5. **A threefold latency penalty.** 0.94 Hz is fine for tabletop bimanual tasks and a real cost for mobile manipulation or dynamic scenes. NativeMEM and TempoFit attack precisely this axis (one token per frame; no new tokens, reuse KV).
6. **20 real trials per task**, no confidence intervals; π_MEM is the authors’ reproduction.

5.3 EventVLA versus TRACE

Both papers address the same situation (the evidence is gone by decision time) and take opposite positions on three axes:

Axis	EventVLA	TRACE
Memory content	Raw image frames	512-D latent slots
Write trigger	Learned future-keyframe probability + NMS + cooldown	Write every step; signature-routed, gated updates decide how much and where
Addressing	Temporal concatenation, attention finds what it needs	Path signatures of the robot-state trajectory as keys
Capacity	5 frames + anchors	K = 4 or 6 fixed slots
Relation to backbone	End-to-end, enters the VLM vision encoder	Adapter; backbone, action head and loss untouched
Inference overhead	Throughput 2.91→0.94 Hz	+2.1 ms per step

The designs match two kinds of information: **high-dimensional visual detail** (colours, positions, several objects) versus **low-dimensional branch variables** (which source, which route). EventVLA's implicit-bank ablation (24.9%) shows the former will not compress into a vector; TRACE's results show the latter fits in a few slots. Which memory to choose depends on how many bits the downstream decision must recover.

6. Related reading

- **KEMO** (2606.23589, same month): event-driven keyframe memory whose events come from robot kinematics plus visual filtering rather than a learned head; keyframes become tokens fused by cross-attention and gated residuals. Real-robot task success +23.6% over $\pi_{0.5}$. Two near-simultaneous papers make event-driven sparse writing the consensus direction of mid-2026.
- **UniMem** (2608.22869): one backbone with an event classifier for memory updates and a keyframe encoder for dense spatial memory; 93.4% vs 68.2% for fixed-interval sampling in simulation.
- **MemoryWAM** (2606.20562): recent frames + event-boundary anchor frames + gist tokens inside a world action model, structurally the same anchors-plus-events layout.
- **MemER** (2510.20328): a high-level VLM selects keyframes in a dual system; used here as the dual-system baseline (10.5% on RoboTwin-MeM).
- **RMBench** (2603.01229) and **RoboMME** (2603.04639): the benchmarks EventVLA argues are solvable by anchors; RoboTwin-MeM fills the transient-evidence gap.
- **Present but Not Remembered** (2607.03372): probing shows history inside frozen VLAs is largely a redundant copy of the present and recommends injecting information unique to the past, which is exactly what sparse event frames do.

---

3. TRACE Deep Dive

> **Paper**: TRACE: Trajectory-Routed Causal Memory for Delayed-Evidence Visuomotor Imitation
> **Authors**: Zihao Li, Ranpeng Qiu, Yincong Chen, Guoqiang Ren, Weiming Zhi (Zhejiang University / Zhejiang University of Technology / University of Sydney)
> **arXiv**: 2606.14551 (2026-06-12, v3) · Project page: jeong-zju.github.io/trace
> **In this repo**: English PDF · Chinese PDF · 中文解读

0. One sentence

TRACE bolts a fixed-capacity latent memory (K = 4 or 6 slots of 512 dimensions) onto ACT and Diffusion Policy and addresses it with **path signatures** of the robot-state trajectory: while a cue is visible, visual-and-state evidence is written into slots selected by "how the robot got here"; when the cue is gone and the robot reaches a visually ambiguous branch point, the same trajectory address reads it back. On five real delayed-evidence tasks, ACT rises from 25.50 to 69.23 mean stage progress and Diffusion Policy from 25.00 to 59.53, above $\pi_{0.5}$ fine-tuned on the same data (50.47); reversing the history order drives route similarity down to 37.8%, showing the memory reads ordered history rather than the current frame.

1. The problem: delayed evidence

The paper isolates a task class: an early cue (object identity, origin shelf, which side a garment came from), a shared execution segment, then a **branch point** where observations under different histories look nearly identical but require different actions. Formally, for a short window `w` there exist histories with

```
x_{t-w+1:t} ≈ x'_{t-w+1:t}   but   a*_t(H_t) ≠ a*_t(H'_t),
```

so `π(a_t | x_t)` must assign one action distribution to two near-identical observations and is wrong at least half the time. What is needed is a compact causal history summary `m_t = m(H_t)` such that `π(a_t | x_t, m_t)` recovers the branch decision through a fixed-size interface.

The critique of the usual remedies is specific. Longer windows fail as soon as the cue leaves the window and force the policy to guess which early frames still matter. Recurrent policies can overwrite or dilute the branch cue, or entangle it with task-progress signals. Generic external memories do not tie writes and reads to the execution history that will be present at the branch point.

2. Method

2.1 Separate memory content from memory address

The central design decision: **the current image and state say what to remember; the executed trajectory decides where it is stored and later retrieved.**

- Content: `e_t = φ_x(x_t)`, pooled multi-view visual features plus a proprioceptive embedding.
- Address: the depth-`p = 3` path signature of the normalised 17-D robot-state path `S_t` (2 base planar velocities, 1 yaw rate, 7 + 7 arm joints),

```
Sig_{≤p}(S_t) = (1, ∫dS, ∫∫_{u1<u2} dS⊗dS, ∫∫∫ dS⊗dS⊗dS).
```

Dropping the scalar term gives `17 + 17² + 17³ = 5219` coordinates, plus a signature-space increment `δ_t = ξ_t − ξ_{t−1}`. Learned projections yield `g_t = φ_g(ξ_t)` and `Δg_t = φ_Δ(δ_t)`, combined into the trajectory key `q_t = φ_q([g_t, Δg_t])`.

Why signatures instead of time indices or another RNN state: signatures are invariant to monotone time reparameterisation (speed does not change the address) and to constant offsets (basepointed), yet sensitive to **order**: replaying the history backwards changes the key. They stream online at 0.52 ms per step. The signature stores no visual cue; it is only the key.

2.2 Signature-routed slot memory

`K` slots `M_t = {m_{t,1}, …, m_{t,K}}`, reset at the start of every episode. Each step:

1. **Route.** `ρ_t = φ_ρ(q_t)` is compared with the previous slot contents (optionally offset by fixed slot identities `η_k` to break symmetry); a softmax gives write weights `ω_{t,k}` (Eq. 6). Routing depends on the trajectory key and on current slot contents, so it is not a fixed hash.
2. **Gated write.** Write proposal `u_t = φ_w(e_t, q_t)`, candidate `m̃_{t,k} = tanh(φ_m(m̄_{t−1,k}, u_t, ρ_t))`, gate `β_{t,k} = ω_{t,k} · σ(φ_β(·))`, update `m_{t,k} = (1−β)m_{t−1,k} + β m̃_{t,k}` (Eqs. 7–8). Slots with small weight barely change; selected slots absorb the current evidence.
3. **Read.** A query from current evidence and routing state attends over the updated slots: `z_t^mem = Σ_k α_{t,k} W_V^r m_{t,k}` (Eq. 9).

The shared state `R_t = (M_t, z_t^mem, g_t, Δg_t)` goes to an adapter. Training and deployment use the same causal scan: only observations and states up to the current step, no cue labels, branch labels, future frames, or test-time demonstration retrieval.

2.3 Adapters translate; they do not change the policy

The updater is shared across policy families; only the policy-facing adapter differs.

- **Regression (ACT) adapter**: each slot becomes a 512-D memory token, `[z_t^mem, g_t, Δg_t]` becomes a summary token, both concatenated to ACT's attention memory. The action head still regresses the chunk with the unchanged L1 (+ optional KL) loss. 0.79 M parameters.
- **Diffusion adapter**: slots pooled by read weights, concatenated with `z_t^mem, g_t, Δg_t`, passed through a zero-initialised MLP into an additive global conditioning vector. Denoising and loss unchanged. 16.03 M parameters.

The shared updater has 11.91 M parameters: +24.6% over ACT (51.62 M), +10.3% over Diffusion Policy (270.96 M).

2.4 Three stabilisers

Finite-slot memories fail in three recognisable ways during training: **slot collapse** (most evidence written through a few slots), **diffuse writing** (one observation spread over all slots, so addresses stop meaning anything), **read–write mismatch** (stored in a form the readout cannot recover). TRACE adds one auxiliary loss for each: a balance loss `L_bal` on uneven average routing, an entropy loss `L_ent` on high-entropy per-step routing, and a consistency loss `L_cons` aligning readout with the write proposal (Eqs. 41–43). They act only on the memory; the imitation loss is untouched. Removing them lowers mean progress from 69.23 to 66.10.

3. Experimental setup

- **Tasks** (collected with the TriPilot-FF whole-body teleoperation system, 30 Hz, one overhead + two wrist cameras, 17-D state):

Task	Early evidence → branch	Demos	Episode length	Subtasks / stages
Tool	Initial object identity → later tool sequence	100	2910 frames (97 s)	2/4
Book	Origin shelf → route and placement	150	1392 frames (46 s)	3/4
Laundry	Origin side → brush-and-basket vs fold-and-store	60	2220 frames (74 s)	2/4
Cable	Origin side → matched device	30	1770 frames (59 s)	1/4
Medicine	Tray origin → box selection and return	60	1759 frames (59 s)	2/4

- **Metric**: stage-level progress (%), 25 physical rollouts per task, task-balanced averages, bootstrap standard errors.
- **Baselines**: ACT and Diffusion Policy (TRACE's two bases); SmolVLA, $\pi_{0.5}$, X-VLA, GR00T N1.6, each adapted for 180k updates on the same single-task demonstrations, with only a generic task prompt (no cue leakage through language).

4. Results

4.1 Main table

Method	Tool	Book	Laundry	Cable	Medicine	Avg.
Diffusion Policy	18.00	12.33	22.00	34.00	38.67	25.00
ACT	31.00	13.67	11.50	44.00	27.33	25.50
$\pi_{0.5}$	45.50	51.00	47.50	49.00	59.33	50.47
GR00T N1.6	39.00	12.67	24.00	38.00	39.33	30.60

Method	Tool	Book	Laundry	Cable	Medicine	Avg.
SmolVLA	25.00	20.00	12.00	50.00	34.67	28.33
X-VLA	5.50	47.00	41.00	36.00	40.00	33.90
TRACE (Regression)	54.50 ± 17.96	**83.00**	**81.00**	**51.00**	**76.67**	**69.23**
TRACE (Diffusion)	**58.50**	68.33	70.50	51.00	49.33	59.53

The largest gains are on Book, Laundry and Medicine, exactly the tasks where an origin cue determines a later branch. Cable gains only +7 because local device geometry near the branch still carries information. Tool has a standard error of 17.96; the authors report a leave-Tool-out check in which TRACE Regression still averages 72.92, 21.21 points above the strongest non-TRACE baseline.

4.2 Against generic memory modules (same ACT base)

Memory module	Online	Fixed budget	Signature	Avg.
No memory	—	—	—	25.50
GRU recurrent memory	✓	✓	—	45.83
Transformer history context	—	—	—	52.10
LRU external memory	✓	✓	—	53.93
Retrieval-prompt memory (MAP-VLA style)	—	—	—	56.30
TRACE signature-routed slots	✓	✓	✓	**69.23**

Any memory beats none (+20 to +30), but the four controls sit within 10 points of each other and TRACE adds 13 more. The authors' explanation: the controls retain history but do not bind the delayed cue to the trajectory address that will be read at the branch point.

4.3 Ablations and diagnostics

Component ablation	Avg.
Current observation only	25.50
Signature only (no stored visual evidence)	45.50
Unrouted slot memory	52.17
No delta routing `Δg_t`	61.43
Mean readout (no attention)	62.80
No auxiliary losses	66.10
Full TRACE	69.23

"Signature only" reaching 45.50 shows the trajectory shape itself carries some branch information (different shelves imply different paths); the remaining 24 points come from storing visual evidence.

The **history-transform diagnostics** are the most persuasive part of the paper. The state path of a recorded online rollout is transformed and the memory module re-run, measuring route similarity (does the pre-branch slot routing sequence match?) and branch consistency (does the branch-point readout and action match?).

Transform	Property tested	Route similarity	Branch consistency
Time resampling	Time-reparameterisation invariance	92.4	96.0
Speed jitter	Same	89.8	94.7
State offset	Offset invariance	91.2	95.1
Sparse sampling	Sampling robustness	86.5	92.3
Order reversal (negative control)	Destroys causal order	**37.8**	**41.6**

Order-preserving transforms stay around 90; reversing the history falls below 40. Same branch-point image, same weights, only the history order changed, and the readout changes with it. That is direct evidence that the memory depends on ordered history, rather than an inference from success rates.

4.4 Long-history stability

Extending online histories to 1.25× / 1.5× / 2.0× or inserting repeated distractor segments keeps slot churn at or below 0.05, with branch decision consistency no lower than 0.872 (repeated distractors). The authors state plainly that this is a diagnostic over the evaluated extensions, not a guarantee for arbitrary horizons.

4.5 Deployment overhead

Path	Base forward	Signature	Slot update	Readout	Adapter	TRACE overhead	Share
Regression	5.90 ms	0.52	0.90	0.58	0.10	2.11 ms	35.8%
Diffusion (100 denoising steps)	118.00 ms	0.52	0.90	0.58	0.14	2.15 ms	1.8%

RTX 5090, PyTorch 2.9.1. Memory is a small fixed cost inside one forward pass, not a second policy call or an offline retrieval.

5. Assessment

5.1 Three things worth remembering

1. **Separating address from content is a new answer to "how should memory be addressed?"** Frame stacking addresses by time, attention retrieval by content, TRACE by the geometry of the executed trajectory. For delayed-evidence tasks this is well-founded: the path the robot has travelled corresponds between cue time and branch time, whereas the images do not.

2. **Diagnostics rather than success rates alone.** Route similarity / branch consistency with an order-reversal negative control is a methodology any memory paper could adopt to answer "what is your memory actually using?". It complements the probing approach of *Present but Not Remembered*.
3. **A plug-in that leaves the backbone alone.** A shared updater and translate-only adapters lift both regression and diffusion families from the same memory state, which is the modularity claim made concrete.

5.2 Points to keep in mind

1. **Signature dimension grows exponentially.** A 17-D state at depth 3 is already 5219-D; depth 4 is 88,740-D. The authors name this the main limitation. Higher-dimensional states (humanoids, dexterous hands) need lower depth or log-signatures (1785-D, but streaming and normalisation are harder; not done here).
2. **Addresses need distinguishable trajectories.** Routing relies on different origins producing different paths. If two branches share identical motion before the branch point (the cue is only a colour, the action sequence identical), signature addresses coincide and routing degrades to content only. All five tasks have origins at different spatial locations, squarely inside TRACE's comfort zone.
3. **Writes happen every step; there is no "should I write?" decision.** Opposite to EventVLA, TRACE writes at every step and lets routing and gates decide how much and where. The stability diagnostics show readout consistency falling to 0.928 at 2× history and 0.907 under repeated distractors: finite slots do get diluted over long histories.
4. **Baseline fairness.** The VLA baselines are fine-tuned for 180k updates with generic prompts but carry no memory module; the only same-base memory comparison is ACT/DP with and without memory. "TRACE + $\pi_{0.5}$" is not answered.
5. **Tool variance is large** (± 17.96). The leave-one-task-out analysis and failure taxonomy handle it honestly, but 25 real rollouts are thin for contact-heavy tasks.
6. **Single-task training.** Each task trains its own policy; there are no multi-task or cross-task memory-transfer results.

5.3 TRACE versus EventVLA

See EventVLA §5.3. In one line: EventVLA stores **many bits of visual detail** (raw frames) and controls sparsity with a learned write trigger; TRACE stores **few bits of branch variable** (latent slots) and controls read/write alignment with trajectory addresses. Both insist that memory be bounded; they are complementary on what to store and how to find it. The natural follow-up: use EventVLA's foresight trigger to decide when to write and TRACE's signatures to decide where, and test whether sparsity and addressability can be had together.

6. Related reading

- **Same group**: TriPilot-FF (2602.09888, the data-collection system), SAI (2606.16490, the third core paper here), AutoIntervene (2608.07065, a visual-action support memory that decides when to hand control to the operator), Tri-Manual (2607.25731).
- **Other slot / fixed-budget latent memories**: ELMUR (2510.07151, layer-local external memory with LRU updates, large gains on MIKASA-Robo), MANN (1605.06065, the origin of content-addressed external memory), VPWEM (2603.04910, out-of-window observations recursively compressed into a fixed number of episodic embeddings).
- **"History should be the policy's state"**: Chronos (2606.30318, selective SSM over the full history, 73.6% on RMBench), TFP (2607.08283, liquid time-constant belief whose write gain is about 6× larger near events), RB-VLA (2602.20659).
- **Path signatures**: Signatory (2001.00706), CILO (2407.04856, signatures encoding trajectory constraints in imitation learning).
- **Source of the retrieval control**: MAP-VLA (2511.09516).

---

4. SAI Deep Dive

> **Paper**: Robots that Collaborate: Sequential Asymmetric Imitation for Learning Coupled Robot Policies
> **Authors**: Yincong Chen, Ranpeng Qiu, Zihao Li, Yanan Zhou, Guoqiang Ren, Weiming Zhi (Zhejiang University / Zhejiang University of Technology / University of Sydney)
> **arXiv**: 2606.16490 (2026-06-15, v2) · Project page: cyc0429.github.io/sai-project-page
> **In this repo**: English PDF · Chinese PDF · 中文解读

0. One sentence

Two bimanual mobile manipulators carry a painting, spread a bed throw and a tablecloth, and collect laundry together, without communicating and without seeing each other fully. SAI collects data with a single teleoperator in three stages: Robot A first learns the task with a compliant human partner; A is then frozen while a human teleoperates B against A's actual behaviour; finally both run together and the operator intervenes on A only where coordination starts to fail. Across four real tasks, success rises from 23–50% (independent imitation) to 53–70%, and the Yield/Wait metric ("wait when the partner is late") from 27–47% to 68–72%. The policy is just an ACT with a 30-step history token and a cascaded base-then-arm head; the coordination comes from the data curriculum, not the architecture.

1. Why this paper belongs in a memory-VLA list

SAI looks like multi-robot imitation learning, but the problem underneath is the same non-Markovian one: **the partner's state is partially observed**. Robot A can only infer from its own cameras and from forces on

the shared object which phase B is in, whether B is delayed, whether B is stuck. That inference has to come from history. Two pieces of evidence in the paper say so directly:

- The 30-step history token (12 log-sampled frames → GRU) cuts premature release in Laundry from 64.5% to 16.1%, because the policy must use history to tell "mid-transport" from "time to release": a single frame looks the same in both.
- Stage-three interventions specifically teach A to slow, wait and recover when "B is not there yet", and whether B is there yet is a latent variable read off a time series.

So SAI is the multi-agent version of memory VLA: what must be remembered is not an occluded object but where a partly visible partner has got to. It comes from the same group as TRACE, shares the policy backbone (ACT with history encoding) and the data-collection system (TriPilot-FF).

2. Problem setting

- Decentralised partially observable control: each robot `i ∈ {A, B}` receives only its local observation `o_i^t = O_i(s_t)` and acts by its own policy `a_i^t ~ π_i(·|o_i^t)`; no messages, partner states, actions or latents are exchanged. Coordination can only arise from local observation of the shared workspace and physical coupling through the shared object.
- Three failure classes: **temporal phase coupling** (grasp, lift, lower, release must align), **partner-contingent yielding** (slow down when the partner is delayed or stuck), **interaction conflict** (excess internal force, deformation, opposing motion, collisions).
- Single-teleoperator constraint: one robot at a time. Supervision is split into three asymmetric datasets: `D_A^solo` (A with a compliant human partner), `D_B^A` (B while A's learned policy runs), `D_A^int` (interventions on A during joint deployment).

The question: are these sequential, asymmetric datasets enough to learn coupled policies that approach what synchronised two-operator demonstrations would give?

3. Method: a three-stage curriculum

3.1 Stage 1: bootstrap A with a human partner

A human supports the other end of the shared object while A is teleoperated. A learns grasping, pulling and aligning. Because the training partner is human and the deployment partner is a robot, naive training lets A shortcut on human appearance (clothes, skin, body shape). The fix is offline **partner-region randomisation**: segment the human, dilate the mask, and at data-loading time replace the region with random RGB noise, heavy Gaussian blur or Stable Diffusion inpainting (Appendix B). The aim is not to erase interaction evidence but to make human appearance unreliable, so the policy attends to object geometry, deformation and contact progress.

3.2 Stage 2: train B against the deployed A

`π_A^(1)` is frozen and deployed; a human teleoperates B. A is run with bounded deployment variation (speed scaling, action noise, timing offsets, perturbed initial object states), so B sees realistic partner deviations and learns to wait, yield, recover and resume. The stage is asymmetric by design: B trains against A's actual learned behaviour rather than an ideal or manually synchronised partner, so it compensates for A's timing, motion profile and residual errors.

3.3 Stage 3: DAgger-style intervention fine-tuning of A

The largest remaining mismatch is on A, which has only ever seen a compliant human. Both policies are deployed together and the operator supplies corrective actions on A only near states where coordination begins to fail (pulling too early, not yielding, continuing while B is delayed, poor recovery after phase mismatch). The interventions are aggregated with the original A demonstrations:

```
π_A^(3) = BC(D_A^solo ∪ D_A^int)
```

Intervention samples are upweighted relative to nominal replay; when only part of the action is corrected, a binary action-dimension mask (e.g., correct the base, keep the arms) prevents sparse labels from overwriting unaffected dimensions. The intervention set is about 30% of the baseline dataset size.

3.4 Policy instantiation

One ACT-style policy per robot: frozen DINOv3-80M (ViT-B) vision, top-view plus wrist cameras, 17-D proprioception (3 base + 14 arm/gripper), arm torque feedback. History window `H = 30` steps, 12 log-sampled frames, pooled multi-view features fused with a 64-D state embedding, a single-layer GRU (hidden 512) whose final state is the history token. The action head is cascaded: an MLP predicts the base action `a_base ∈ R³ = (v_x, v_y, ω)`, which is concatenated with the decoder query to predict the 14-D arm/gripper action. Chunk length 100, temporal ensembling coefficient 0.01.

4. Experiments

4.1 Four real tasks, three training pipelines

Tasks: Bed-throw Spreading and Tablecloth Spreading (deformable alignment), Laundry Collection (asynchronous coordination in a shared workspace), Painting Transport (contact-rich rigid transport). Three pipelines share one architecture: Independent Imitation (each robot from its own unilateral demos), Partner-Conditioned Imitation (stage 2 without stage 3), full SAI. 30 rollouts per task-method pair; Phase Sync counts predefined checkpoints (Painting: 5 × 30 = 150), Yield/Wait counts partner-delay opportunities.

Task	Method	Success	Phase Sync	Yield/Wait
Bed-throw	Independent / Partner-cond. / **SAI**	23.3 / 36.7 / **53.3**	30.8 / 60.8 / **62.5**	26.7 / 35.0 / **68.3**
Tablecloth	Independent / Partner-cond. / **SAI**	43.3 / 60.0 / **66.7**	54.4 / 67.8 / **68.9**	46.7 / 61.7 / **70.0**

Task	Method	Success	Phase Sync	Yield/Wait
Laundry	Independent / Partner-cond. / **SAI**	50.0 / 63.3 / **70.0**	51.7 / 68.3 / **73.3**	31.7 / 48.3 / **71.7**
Painting	Independent / Partner-cond. / **SAI**	33.3 / 46.7 / **56.7**	42.7 / 67.3 / **74.7**	34.4 / 48.9 / **68.9**

How to read it: stage 2 mainly lifts Phase Sync (Bed-throw 30.8→60.8) because B learns A's rhythm; stage 3 mainly lifts Yield/Wait (Bed-throw 35.0→68.3) because A finally learns to wait when B falls behind. Success is the sum of the two.

4.2 Controlled partner delay

In Tablecloth, B is manually paused. Independent-imitation A keeps pulling on its nominal trajectory, the cloth misaligns and B loses its grasp; SAI's A sees that B has not advanced, slows and waits, then resumes when B recovers. That is the behaviour behind the Yield/Wait number.

4.3 Architecture ablations: history and cascade

Failure mode	Variant	Rate
Premature release (Laundry delivery phase)	without / with history	64.5% / **16.1%**
Premature lowering (transport→place transition)	without / with cascade	51.6% / **9.7%**

The history token lets the policy separate mid-transport from terminal release; the cascaded head conditions arm commands on base intent so the arm does not lower before the base arrives. The authors stress that these components improve local execution reliability and **do not** explain the collaborative gains: the three pipelines in Table 1 share the architecture and independent imitation still fails under partner delay.

4.4 Backbone independence

On Painting Transport with a Diffusion Policy head: Independent 23.5 / 35.2 / 30.7 → SAI 52.9 / 62.2 / 76.9 (Success / Phase Sync / Yield/Wait, 34 rollouts). With ACT: 33.3 / 42.7 / 34.4 → 56.7 / 74.7 / 68.9. The curriculum's effect does not depend on the action decoder.

5. Assessment

5.1 Worth remembering

1. **Coordination is a function of the partner distribution, not the architecture.** The experimental design fixes the architecture and changes only the data curriculum, so the gains are cleanly attributable to what partners the policy saw during training. It is the same experimental discipline as TRACE's plug-in memory with an untouched backbone.
2. **Asymmetry is a feature.** B learns against the real A; A then patches against the real B. Each stage fixes the currently largest distribution mismatch. Synchronised two-operator data is expensive and not

obviously better, since it trains against a perfect partner.

3. **History's role in multi-robot control is quantified.** Premature release 64.5% → 16.1% is a direct instance of history disambiguating visually identical states, structurally the same as TRACE's delayed-evidence setting.

5.2 Points to keep in mind

1. **Intervention coverage is thin.** Stage-3 data is about 30% of the baseline set and the authors say recovery "has not saturated"; too much correction could bias the policy toward needless waiting. How many interventions and where remains a human judgement. The group's follow-up AutoIntervene (2608.07065) addresses exactly this with a visual-action support memory and quantile-calibrated switching thresholds.
2. **A-centric.** Only A is corrected. Tasks needing simultaneous correction of both robots do not fit; B is never updated after stage 2, although the distribution it faces changes once A changes. There is no stage 4.
3. **History is 30 steps, about a second.** Enough to separate adjacent phases, not enough to know what B did two minutes ago. Partner phase over longer tasks may need TRACE-style explicit slots, which is the most natural fusion of the two papers.
4. **Descriptive metrics.** Phase Sync and Yield/Wait denominators are event counts, not independent trials (Painting's 150 checkpoints come from 30 rollouts), so they cannot be treated as independent samples; the appendix says so.
5. **Homogeneous robots** (two identical bimanual mobile manipulators with TriPilot-FF). Whether human-appearance randomisation suffices with heterogeneous partners (humanoid plus wheeled, as in Duet) is untested.
6. **No comparison to policies trained on synchronised two-operator demonstrations**, the very target SAI is meant to replace; the cost of collecting them is the reason, so "approaches synchronised demonstrations" remains a hypothesis.

5.3 Relation to the other two papers

	EventVLA	TRACE	SAI
What is unobservable	Occluded or vanished object evidence	An early branch cue	The partner's internal phase
What fills the gap	Sparse raw-frame buffer	Trajectory-addressed latent slots	30-step GRU history + training-distribution coverage
Where the intervention sits	Architecture (KEM head)	Module (plug-in adapter)	Data (curriculum + DAgger)
Shared stance	All three reject "just lengthen the window" and use a bounded history representation		

6. Related reading

- **Same group**: TRACE, TriPilot-FF, AutoIntervene, Tri-Manual (2607.25731: one operator demonstrates for three arms; demonstrations are re-timed offline under dependency constraints).
- **Multi-robot data collection**: HATS (2606.16491, the adjacent arXiv number: one human plus an MLLM agent controlling four arms), Duet (2606.20990, human–human demonstrations for pretraining plus a few dual-robot teleoperation trajectories, Unitree G1 + Vega1 heterogeneous partners), Mobile ALOHA (2401.02117).
- **Interventional imitation**: DAgger (1011.0686), IntervenGen (2405.01472, generating many corrective interventions from few human ones; a direct remedy for SAI's stage-3 coverage).
- **Decoupled bimanual control**: Decoupled Interaction Framework (2503.09186, one model per arm plus a selective interaction module, +23.5% on RoboTwin), the single-robot version of "decentralised with local interaction".
- **Making two action heads agree** (2608.15748): coordination mechanisms and a runtime collapse certificate for dual-branch flow-matching policies; a formal reference for the failure where two robots pick different modes.

---

5. Design-Space Matrix

每个记忆 VLA 都要回答六个问题：**存什么**（内容）、**何时写**（触发）、**写到哪 / 从哪读**（寻址）、**存多少**（容量）、**接在哪**（集成点）、**靠什么学**（监督）。下表把三篇核心论文和 30 个代表性工作填进这六列，条目内容只取自原文或摘要。方法名后的括号是 arXiv 编号。横向解读见趋势与洞见 §2。

方法	存什么	何时写	寻址	容量	集成点	监督	主要数字
EventVLA (2606.20092)	原始图像帧	学习的未来关键帧概率 ≥0.55 + NMS + 冷却	时间顺序拼接，注意力自找	5 事件帧 + 初始帧 + 2–3 近期帧	VLM 视觉编码器输入	Qwen3-VL 自动标签 + 升余弦软标签 + 师生课程	RoboTwin-MeM 18.0→75.2；RMBench 67.8（仅锚帧）
TRACE (2606.14551)	视觉 + 状态的 512 维 latent	每步，门控决定写多少	机器人轨迹路径签名（深度 3，5219 维）+ 增量	K = 4/6 槽	动作头前适配器（ACT 记忆 token / DP 全局条件）	无标签；平衡 + 熵 + 一致性稳定器	ACT 25.5→69.2；顺序反转路由相似度 37.8
SAI (2606.16490)	GRU 隐状态（12 帧对数采样）	每步	无	30 步	ACT 解码器输入	三阶段数据课程 + DAgger 干预	提前松手 64.5→16.1%；成功 23–50→53–70%

方法	存什么	何时写	寻址	容量	集成点	监督	主要数字
KEMO (2606.23589)	关键帧 token	运动学 + 视觉过滤检测事件	时序 token，交叉注意力	事件数	门控残差融合进 VLA	事件附近样本加权	真机 +23.6 成功 / +34.1 阶段
UniMem (2608.22869)	关键帧密集空间记忆	事件分类器	注意力	关键帧缓存	单骨干	分类器监督	仿真 93.4 vs 68.2
WeaveLA (2606.17463)	完成段的 latent token	子目标完成事件	直接路由进下一子任务	每子任务若干 token	动作生成路径（冻结 VLA）	模仿损失	SwingXtimes N=3：0→47.8
Keyframe-Chaining (2603.01465)	关键帧作交错视觉 token	判别式嵌入选帧	进度感知查询	若干关键帧	VLA 输入	对比嵌入	ManiSkill 4 任务
MemER (2510.20328)	关键帧 + 文本指令	高层 VLM 选帧	VLM 推理	选中帧 + 近期帧	双系统（Qwen2.5-VL-7B → $\pi_{0.5}$）	少量语言标注	分钟级真机任务
MEM / π_MEM (2603.03596)	视频编码短时 + 文本长时	持续	文本检索 / 视频窗口	文本无界	双系统	演示	15 分钟级任务
MemoryVLA (2508.19236)	感知 + 认知 token 库	每步，冗余合并	内容相似检索	库大小	记忆条件扩散专家	模仿损失	MIKASA-Robo 41.2
ContextVLA (2510.04246)	单个上下文 token	每步	—	1 token	VLM 输入	模仿损失	多帧收益、低成本
HAMLET (2510.00695)	moment token	每步	轻量记忆模块	历史长度	GR00T N1.5	时间对比初始化	真机 76.4（+47.2）
NativeMEM (2607.06678)	每帧每视角 1 token	每帧	序列位置	线性增长（token 极小）	追加到 VLA 输入	冻结 VLA 监督的 tokenizer → 解冻微调	仿真 32.4→84.0
TempoFit (2603.07647)	中间层前缀 K/V	每步 FIFO	K-to-K 检索 + 帧间隙偏置	层内 FIFO	注意力前残差	免训练	LIBERO-LONG +4.0
StreamPI (2608.26067)	（观测，指令）对的 KV	每帧	因果注意力	随流增长	骨干注意力掩码	随机间隔流式训练	零新增参数
DySta (2602.03983)	静态 / 动态 token	静态 token 仅在需要时重缓存	KV 复用	一份静态 + 动态	VLM 输入	模仿损失	2 倍加速 +2.3

方法	存什么	何时写	寻址	容量	集成点	监督	主要数字
PRISM (2606.16178)	分层压缩 token	每步	门控注意力	两分钟	Transformer 策略	模仿损失	ReMemBench +5–12
HALO (2606.25136)	长上下文 token	每步	稀疏注意力检索	八分钟	Transformer 策略	VLM 生成的记忆问答蒸馏	—
AtlasVLA (2608.06729)	4D 体素哈希空间状态 + 自我状态	每帧更新	空间哈希	场景体素	DiT 条件	模仿损失	LIBERO-Long +9.4
LaMem-VLA (2607.07608)	短期 / 长期 latent token	curator 整理	seeker 多模态查询	有界上下文	编织进嵌入序列	模仿损失	SimplerEnv / LIBERO
μVLA (2606.12497)	m 个记忆 token	每步自注意力更新	隐式	m 个 token	骨干内	TBPTT，无辅助损失	MIKASA 0.42→0.84
VPWEM (2603.04910)	情景记忆嵌入	窗外观测递归压缩	交叉注意力	固定数量	扩散策略条件	模仿损失	MIKASA +20%
VQ-Memory (2603.09513)	离散 VQ token（本体）	每步	序列	码本	VLA / DP 条件	VQ-VAE	RuleSafe
MemoAct (2603.18494)	感觉 / 短期无损 / 长期压缩	分层	分层	三层	策略条件	模仿损失	MemoryRTBench
Chronos (2606.30318)	每控制步一个状态 token	每步	选择性 SSM	全历史（压缩）	策略 latent 状态	IMLE + 薛定谔桥	RMBench 73.6
TFP (2607.08283)	进度信念（LTC 动力学）	每步，事件附近增益 ~6×	—	单信念	流匹配解码器调制	模仿损失	MIKASA ShellGameTouch 75.0
CAMP (2606.21188)	压缩的动作历史	每步	—	固定	策略条件	自监督（动作历史）	Memory-T-Bench
GMP (2604.18933)	latent 记忆	记忆门决定何时回忆	交叉注意力	固定	视觉运动策略	历史动作加扩散噪声	MemMimic +30.1
ELMUR (2510.07151)	层内外部记忆嵌入	每段	LRU 替换 / 凸混合	每层固定	Transformer 各层	RL 目标	MIKASA 21/23
RB-VLA (2602.20659)	紧凑信念状态	每步	—	固定	扩散策略条件	自监督世界模型	32.5→77.5
FM-VLA (2607.18231)	力记忆 token	每步	—	短历史	动作专家条件	VAE 力重建	80%+

方法	存什么	何时写	寻址	容量	集成点	监督	主要数字
AGM (2608.29537)	子目标序列 + 进度指针	物理证据验证后才推进	指针	子目标数	冻结 VLA 外部	2.43M 验证头	RoboMME Counting 领先
HyMeS (2608.09410)	代码里的记忆结构	阶段完成验证	代码逻辑	无界	编码智能体 → VLA	rollout 反馈迭代启发式	RoboMemArena 52.5→66.2
PonderPounce (2608.24115)	MLLM 原生因果上下文	持续	注意力	上下文长度	异步认知 token → VLA	端到端联合训练	RoboMME 60.83
BATON (2608.16889)	子任务解 + 转移记忆	探索后写入	子任务索引	子任务数	LLM 智能体 + 冻结 VLA	无参数更新	RoboMemArena +11.6
Notes-to-Self (2602.21013)	语言草稿板	模型自写	文本	文本	VLA 提示	演示	ClevrSkills / MemoryBench
MemoryWAM (2606.20562)	近期帧 + 事件边界锚帧 + gist token	事件边界	定制注意力	三层有界	世界动作模型	视频 + 动作	长时程记忆任务
MemoryVAM (2606.20679)	Perceiver 压缩的记忆 token	每帧	交叉注意力	固定	视频骨干 + 动作解码器	视频预测 + 回合边界监督	LIBERO-Mem 5→42.5

三个空白（读表可见）

1. **「何时写」和「寻址」同时学的方法还没有出现**：EventVLA 学写入时机但按时间寻址，TRACE 按轨迹寻址但每步都写。
2. **多机器人一列几乎为空**：SAI 只有 30 步 GRU，没有显式记忆模块用于搭档状态估计。
3. **监督来源五花八门**（235B 标注 / 规则 / 分类器 / rollout 结果 / 自监督），没有跨任务对照说明哪种写入策略能迁移。

---

6. Annotated Paper List

Groups follow the README. Each group opens with a positioning paragraph, followed by title, authors, arXiv id and the first sentences of the abstract for every paper.

Core Papers (Deep Dives)

The repository is organised around these three papers from June 2026. They answer one question, what a policy can rely on when the evidence is gone by decision time, at three different levels: architecture (EventVLA's keyframe evidence memory), module (TRACE's trajectory-addressed slot memory) and data

(SAI's partner-distribution curriculum). Each has a detailed report in Chinese and English, the original PDF and a layout-preserving Chinese translation.

1. **EventVLA: Event-Driven Visual Evidence Memory for Long-Horizon Vision-Language-Action Policies** (arXiv 2026, arXiv 2606.20092). *Ganlin Yang, Zhangzheng Tu, Yuqiang Yang, Sitong Mao, Junyi Dong, Tianxing Chen, Jiaqi Peng, Jing Xiong et al.*

   Memory remains a critical bottleneck for long-horizon robotic manipulation, as standard Vision-Language-Action (VLA) policies often fail when task-relevant cues become occluded or unobservable over time. While existing memory-augmented methods utilize historical context, they either suffer from severe information bottlenecks, incur high latency via decoupled dual systems, or rely on unselective buffers that accumulate massive visual redundancies.

2. **TRACE: Trajectory-Routed Causal Memory for Delayed-Evidence Visuomotor Imitation** (arXiv 2026, arXiv 2606.14551). *Zihao Li, Ranpeng Qiu, Yincong Chen, Guoqiang Ren, Weiming Zhi*

   Robots under autonomous operation may require decisions based on evidence that is no longer visible. We study delayed-evidence tasks, where an early cue disappears before a later decision point, so visually similar observations can require different actions.

3. **Robots that Collaborate: Sequential Asymmetric Imitation for Learning Coupled Robot Policies** (arXiv 2026, arXiv 2606.16490). *Yincong Chen, Ranpeng Qiu, Zihao Li, Yanan Zhou, Guoqiang Ren, Weiming Zhi*

   Collaborative mobile manipulation requires robots to coordinate with a partially observed partner while physically interacting through shared objects. This is difficult because failures often arise not from poor local skills, but from mistimed waiting, yielding, pulling, releasing, or repositioning.

Surveys and Analyses

Read the three analyses before the methods: Present but Not Remembered shows with probes and causal interventions that history inside frozen VLAs is largely a redundant copy of the present; the long-context diffusion-policy study shows naive context scaling is less brittle than advertised; WhyChunking shows that one benefit of action chunking is precisely non-Markovian expressivity.

1. **Weights or Skills? A Survey of Robot-Learning Techniques: from Action-Predicting Weights to Robots that Write their Own Skills** (arXiv 2026, arXiv 2608.01851). *Gaytri Jena, Kapil Wanaskar, Vinija Jain, Aman Chadha, Vasu Sharma, Amitava Das*

   Robot learning is splitting into two bets: policies that bake competence into frozen weights (vision-language-action, or VLA, models), and agents that write and refine their own executable skills as code. This survey organises the field around that axis of weights versus skills.

2. **Why Does Action Chunking Improve Behavioral Cloning Performance in Robotic Control?** (arXiv 2026, arXiv 2608.02547). *Filippo Lazzati, Kyle Stachowicz, William Chen, Alberto Maria Metelli, Andrew Wagenmaker, Sergey Levine*

Action chunking—predicting and executing multiple actions instead of a single action—has proven to be a critical component for learning effective robotic control policies. However, our precise understanding of why action chunking improves performance has remained limited.

3. **Present but Not Remembered: Auditing How Frozen VLAs Encode, Deploy, and Steer Visual History** (arXiv 2026, arXiv 2607.03372). *Chih-Ting Liao, Xin Cao*

   A frozen vision-language-action model (VLA) receives recent observations at every decision step, yet prior work has focused on adding memory rather than asking how existing history is represented and used. We study this temporal axis using layer-resolved linear probing and causal interchange interventions across three VLAs from two architecture families.

4. **World Action Models: A Survey** (arXiv 2026, arXiv 2606.20781). *Qiuhong Shen, Shihua Zhang, Yue Liao, Qi Li, Zhenxiong Tan, Shizun Wang, Shuicheng Yan, Xinchao Wang*

   World Action Models (WAMs) are embodied predictive-action models that make a forecast of the future available to action. Recent WAMs repurpose large video generation models, and a parallel line relies on language or vision-language backbones without a video-generation core.

5. **Training and Evaluating Diffusion Policies with Long Context Lengths** (arXiv 2026, arXiv 2606.16447). *Abhinav Agarwal, Adam Wei, Taylan Kargin, Michael Zeng, Cole Becker, Arif Kerem Dayi, Pablo Parrilo, Asuman Ozdaglar et al.*

   Imitation learning has enabled highly-dexterous robotic manipulation from RGB observations. Policies trained with these methods, however, typically condition robot actions on only a short history of observations.

6. **Large VLM-based Vision-Language-Action Models for Robotic Manipulation: A Survey** (arXiv 2025, arXiv 2508.13073). *Rui Shao, Wei Li, Lingsen Zhang, Renshan Zhang, Zhiyang Liu, Ran Chen, Liqiang Nie*

   Robotic manipulation, a key frontier in robotics and embodied AI, requires precise motor control and multimodal understanding, yet traditional rule-based methods fail to scale or generalize in unstructured, novel environments. In recent years, Vision-Language-Action (VLA) models, built upon Large Vision-Language Models (VLMs) pretrained on vast image-text datasets, have emerged as a transformative paradigm.

Benchmarks for Memory-Dependent Manipulation

RMBench, RoboMME and RoboMemArena landed in spring 2026 and became the shared target of the summer's method papers. RoboMME's conclusion that memory representations are highly task-dependent has been confirmed by a year of results. EventVLA's own RoboTwin-MeM parameterises how many intermediate keyframes must be retained (n) and is the only benchmark that isolates transient evidence; benchmarks embedded in method papers (ReMemBench, LIBERO-Mem, MemMimic, MemoryRTBench, RuleSafe, Memory-T-Bench, MemoryBench) are listed under their papers.

1. **RoboDojo: A Unified Sim-and-Real Benchmark for Comprehensive Evaluation of Generalist Robot Manipulation Policies** (arXiv 2026, arXiv 2607.04434). *Tianxing Chen, Yue Chen, Zixuan Li, Junyuan Tang, Kailun Su, Haoran Lu, Weijie Wan, Baijun Chen et al.*

   Generalist robot manipulation policies have advanced rapidly, yet existing benchmarks remain limited in systematically evaluating their capabilities. Many rely on simple, short-horizon, or skill-narrow tasks with limited capability coverage, and are often conducted only in simulation or only in the real world.

2. **RoboMemArena: A Comprehensive and Challenging Robotic Memory Benchmark** (arXiv 2026, arXiv 2605.10921). *Huashuo Lei, Wenxuan Song, Huarui Zhang, Jieyuan Pei, Jiayi Chen, Haodong Yan, Han Zhao, Pengxiang Ding et al.*

   Memory is a critical component of robotic intelligence, as robots must rely on past observations and actions to accomplish long-horizon tasks in partially observable environments. However, existing robotic memory benchmarks still lack multimodal annotations for memory formation, provide limited task coverage and structural complexity, and remain restricted to simulation without real-world evaluation.

3. **LongBench: Evaluating Robotic Manipulation Policies on Real-World Long-Horizon Tasks** (arXiv 2026, arXiv 2604.16788). *Xueyao Chen, Jingkai Jia, Tong Yang, Yibo Fu, Wei Li, Wenqiang Zhang*

   Robotic manipulation policies often degrade over extended horizons, yet existing benchmarks provide limited insight into why such failures occur. Most prior benchmarks are either simulation-based or report aggregate success, making it difficult to disentangle the distinct sources of temporal difficulty in real-world execution.

4. **RoboMME: Benchmarking and Understanding Memory for Robotic Generalist Policies** (arXiv 2026, arXiv 2603.04639). *Yinpei Dai, Hongze Fu, Jayjun Lee, Yuejiang Liu, Haoran Zhang, Jianing Yang, Chelsea Finn, Nima Fazeli et al.*

   Memory is critical for long-horizon and history-dependent robotic manipulation. Such tasks often involve counting repeated actions or manipulating objects that become temporarily occluded.

5. **RMBench: Memory-Dependent Robotic Manipulation Benchmark with Insights into Policy Design** (arXiv 2026, arXiv 2603.01229). *Tianxing Chen, Yuran Wang, Mingleyang Li, Yan Qin, Hao Shi, Zixuan Li, Yifan Hu, Yingsheng Zhang et al.*

   Robotic manipulation policies have made rapid progress in recent years, yet most existing approaches give limited consideration to memory capabilities. Consequently, they struggle to solve tasks that require reasoning over historical observations and maintaining task-relevant information over time, which are common requirements in real-world manipulation scenarios.

6. **RoboCerebra: A Large-scale Benchmark for Long-horizon Robotic Manipulation Evaluation** (NeurIPS 2025, arXiv 2506.06677). *Songhao Han, Boxiang Qiu, Yue Liao, Siyuan Huang, Chen Gao, Shuicheng Yan, Si Liu*

Recent advances in vision-language models (VLMs) have enabled instruction-conditioned robotic systems with improved generalization. However, most existing work focuses on reactive System 1 policies, underutilizing VLMs' strengths in semantic reasoning and long-horizon planning.

7. **Memory, Benchmark & Robots: A Benchmark for Solving Complex Tasks with Reinforcement Learning** (arXiv 2025, arXiv 2502.10550). *Egor Cherepanov, Nikita Kachaev, Alexey K. Kovalev, Aleksandr I. Panov*

   Memory is crucial for enabling agents to tackle complex tasks with temporal and spatial dependencies. While many reinforcement learning (RL) algorithms incorporate memory, the field lacks a universal benchmark to assess an agent's memory capabilities across diverse scenarios.

Event and Keyframe Memory

Write only when something happened; store raw frames or their tokens. The consensus direction of mid-2026: EventVLA learns future-keyframe probabilities, KEMO uses kinematic and visual rules, UniMem trains an event classifier, WeaveLA fires on subgoal completion, Keyframe-Chaining retrieves with progress-aware queries.

1. **UniMem: Unifying Multimodal Memory and Control for Vision-Language-Action Models** (arXiv 2026, arXiv 2608.22869). *Lars Osterberg, Maggie Wang, Mac Schwager*

   While Vision-Language-Action (VLA) models have leveraged internet-scale pretraining and task-focused finetuning to achieve strong performance on long-horizon tasks, they often struggle with non-Markovian tasks that require memory. Existing approaches to memory typically involve additional Vision-Language-Models (VLMs) for long-term memory management, introducing a memory bottleneck and a fractured training pipeline.

2. **KEMO: Event-Driven Keyframe Memory for Long-Horizon Robot Manipulation with VLA Policies** (arXiv 2026, arXiv 2606.23589). *Yihan Zeng, Minghao Ye, Yiyuan Chen, Yide Shentu, Philipp Wu, Zike Yan, Zhongyu Li*

   Long-horizon robot manipulation remains challenging because similar observations may occur at different execution stages, while the appropriate action depends on previously completed operations. Memory can address this ambiguity by enabling policies to infer task progress from execution history.

3. **WeaveLA: Event Driven Cross-Subtask Latent Memory Weaving for Repetitive Robot Manipulation** (arXiv 2026, arXiv 2606.17463). *Shoujing Zhu, Zhenyang Liu, Fungmiu Wang, Jiafeng Wang, Bo Yue, Guiliang Liu, Simo Wu, Xiangyang Xue et al.*

   Vision-Language-Action (VLA) policies have achieved remarkable single-step manipulation, yet they remain brittle precisely where each stage depends on what was just completed. The core issue is structural: short-window VLAs lack an explicit channel for rouxting information across sub-task boundaries, and existing memory-augmented variants either write at every frame, retrieve from demonstration-time stages, or fire at sub-goal events without performing an explicit sub-task-to-sub-task hand-off into the action expert.

4. **Bi-HIL: Bilateral Control-Based Multimodal Hierarchical Imitation Learning via Subtask-Level Progress Rate and Keyframe Memory for Long-Horizon Contact-Rich Robotic Manipulation** (arXiv 2026, arXiv 2603.13315). *Thanpimon Buamanee, Masato Kobayashi, Yuki Uranishi*

   Long-horizon contact-rich robotic manipulation remains challenging due to partial observability and unstable subtask transitions under contact uncertainty. While hierarchical architectures improve temporal reasoning and bilateral imitation learning enables force-aware control, existing approaches often rely on flat policies that struggle with long-horizon coordination.

5. **Non-Markovian Long-Horizon Robot Manipulation via Keyframe Chaining** (arXiv 2026, arXiv 2603.01465). *Yipeng Chen, Wentao Tan, Lei Zhu, Fengling Li, Jingjing Li, Guoli Yang, Heng Tao Shen*

   Existing Vision-Language-Action (VLA) models often struggle to generalize to long-horizon tasks due to their heavy reliance on immediate observations. While recent studies incorporate retrieval mechanisms or extend context windows to handle procedural tasks, they often struggle to capture Non-Markovian dependencies, where optimal actions rely solely on specific past states rather than the current observation.

Dense and Compressed Visual History

Every frame enters; cost is controlled by compression, tokenisation, KV reuse or sampling. The recent trend here is nearly-free memory: NativeMEM at one token per frame, TempoFit reusing K/V with no training, StreamPI with zero new parameters, DySta reusing the KV cache of static tokens. EventVLA's threefold latency is the problem this family is solving.

1. **StreamPI: Streaming Multimodal Temporal Modeling for Vision-Language-Action Models** (arXiv 2026, arXiv 2608.26067). *Zhe Liu, Jinghua Hou, Yuxiang Lu, Zhenya Yang, Xianzhe Fan, Junwei Luo, Junyi Li, Ruihua Han et al.*

   Vision-Language-Action (VLA) models have demonstrated effectiveness in robot manipulation, yet state-of-the-art models such as pi0.5 operate under a single-frame paradigm, limiting their ability to retain past observations and develop precise spatial perception. In this paper, we propose StreamPI, a streaming multimodal temporal modeling framework that equips single-frame VLA with temporal reasoning capability without introducing any additional parameters.

2. **Remember Smarter: Visual History Compressor and Hyperbolic Experience Space for Robotic Memory** (arXiv 2026, arXiv 2608.15269). *Dai Zhou, Jiexi Yan, Tong Li, Yuxuan Wang, Cheng Deng*

   Long-horizon robot policies require compact access to recent observations and reusable experience without expanding the vision-language-action (VLA) context. We introduce Remember Smarter (RS), a plug-and-play module with complementary visual-history and hyperbolic experience-memory branches.

3. **AtlasVLA: Persistent World-Ego State Modeling for Vision-Language-Action Models** (arXiv 2026, arXiv 2608.06729). *Guiyu Zhao, Longteng Guo, Yanghong Mei, Zilin Zhu, Yu Zhang, Bin Cao, Mingming Yu, Xingjian He et al.*

While Vision-Language-Action (VLA) models have advanced embodied AI, their fundamentally reactive paradigm severely limits performance in partially observable and long-horizon tasks. When restricted to a single wrist-mounted camera, they inevitably suffer from perception forgetting as objects exit the field of view, and temporal task-progress forgetting} during multi-step execution.

4. **BridgeVLA++: A Data-Efficient, Generalizable, and Memory-Augmented Vision-Language-Action Framework for 3D Manipulation** (arXiv 2026, arXiv 2608.05042). *Peiyan Li, Yuze Zhu, Yixiang Chen, Qisen Ma, Yuan Xu, Jiabing Yang, He Guan, Yan Huang et al.*

   Leveraging pre-trained vision-language models (VLMs) to construct vision-language-action (VLA) models has emerged as a promising paradigm for 3D robot manipulation. However, existing 3D VLA methods remain data-hungry, exhibit limited generalization under distribution shifts, and lack explicit memory of past observations.

5. **FibVLA: An Efficient Temporal Vision-Language-Action Model with Fibonacci Sampling** (arXiv 2026, arXiv 2607.29596). *Li Lin, Wujun Xu, Weiwei Meng, Kaiwen Xia, Kang Hao Cheong, Shuai Wang*

   Vision-language-action models (VLAs), which leverage the cognition of multimodal information to infer physical-world actions, provide a generalized solution for embodied AI applications. Conventional VLAs usually concentrate on current digital cognition.

6. **Dual Latent Memory in Vision-Language-Action Models for Robotic Manipulation** (arXiv 2026, arXiv 2607.07608). *Hongyu Qu, Jianzhe Gao, Xiaobin Hu, Shaohuan Yang, Xinlei Yu, Rui Yan, Wenguan Wang, Xiangbo Shu et al.*

   Mainstream Vision-Language-Action (VLA) models predict actions primarily from the current observation under a Markovian assumption, thus struggling with long-horizon, temporally dependent tasks. Existing memory-augmented VLAs either expand the observation window or retrieve history from the memory bank as auxiliary policy-side context.

7. **NativeMEM: Native Memory Compression for Long-Horizon Robotic Manipulation** (arXiv 2026, arXiv 2607.06678). *Ziye Wang, Modi Shi, Chaojun Ni, Jiazhi Yang, Mengdi Li, Zhizhong Su, Tianwei Lin, Hongyang Li*

   How can pretrained Vision-Language-Action (VLA) models retain long-horizon visual histories with high-frequency updates without sacrificing efficiency? Existing approaches rely on external memory management, which restrains either the memory horizon or the reactiveness of pretrained policies.

8. **HiMe: Hierarchical Embodied Memory for Long-Horizon Vision-Language-Action Control** (arXiv 2026, arXiv 2607.03449). *Li Ji, Siyin Wang, Pengfang Qian, Xiaopeng Yu, Yihai Tian, Zhaoye Fei, Jingjing Gong, Xipeng Qiu*

   Current Vision-Language-Action (VLA) models excel at robotic manipulation but often struggle with non-Markovian tasks requiring long-term memory and reasoning due to their reliance on immediate observations. Existing solutions face a ‘’frequency-competence paradox,‘‘ where stronger reasoning models are too slow for real-time control, while faster models lack sufficient reasoning capabilities.

9. **Memory Retrieval in Visuomotor Policies for Long-Horizon Robot Control** (arXiv 2026, arXiv 2606.25136). *Rutav Shah, Yisu Li, Femi Bello, Yuke Zhu, Roberto Martín-Martín*

   General-purpose robots operating in partially observable environments, such as homes, require memory to support autonomy. They must recall diverse information from the past, such as where objects were placed, which tasks a human partner has completed, and when an appliance was turned on.

10. **Scaling Short-Term Memory of Visuomotor Policies for Long-Horizon Tasks** (arXiv 2026, arXiv 2606.16178). *Rutav Shah, Rajat Kumar Jenamani, Xiaohan Zhang, Lingfeng Sun, Roberto Martín-Martín, Yuke Zhu, Deva Ramanan, Karl Schmeckpeper*

    Many robotic tasks require short-term memory, whether it's retrieving an object that's no longer visible or turning off an appliance after a set period. Yet, most visuomotor policies trained via imitation learning rely only on immediate sensory input without using past experiences to guide decisions.

11. **MemoryVLA++: Temporal Modeling via Memory and Imagination in Vision-Language-Action Models** (arXiv 2026, arXiv 2606.09827). *Hao Shi, Weiye Li, Bin Xie, Yulin Wang, Renping Zhou, Tiancai Wang, Xiangyu Zhang, Ping Luo et al.*

    Temporal modeling is essential for robotic manipulation, as effective control requires both memory of past interactions and imagination of future states. However, most VLA models rely primarily on the current observation and therefore struggle with long-horizon, temporally dependent tasks.

12. **ST-VLA: Enabling 4D-Aware Spatiotemporal Understanding for General Robot Manipulation** (arXiv 2026, arXiv 2603.13788). *You Wu, Zixuan Chen, Cunxu Ou, Wenxuan Wang, Wenbo Huang, Lin Cao, Yangtao Chen, Weichao Qiu et al.*

    Robotic manipulation in open-world environments requires reasoning across semantics, geometry, and long-horizon action dynamics. Existing hierarchical Vision-Language-Action (VLA) frameworks typically use 2D representations to connect high-level reasoning with low-level control, but lack depth awareness and temporal consistency, limiting robustness in complex 3D scenes.

13. **AnchorVLA4D: an Anchor-Based Spatial-Temporal Vision-Language-Action Model for Robotic Manipulation** (arXiv 2026, arXiv 2603.12730). *Juan Zhu, Zhanying Shao, Xiaoqi Li, Ethan Morgan, Jiadong Xu, Hongwei Fan, Hao Dong*

    Since current Vision-Language-Action (VLA) systems suffer from limited spatial perception and the absence of memory throughout manipulation, we investigate visual anchors as a means to enhance spatial and temporal reasoning within VLA policies for robotic manipulation. Conventional VLAs generate actions by conditioning on a single current frame together with a language instruction.

14. **TempoFit: Plug-and-Play Layer-Wise Temporal KV Memory for Long-Horizon Vision-Language-Action Manipulation** (arXiv 2026, arXiv 2603.07647). *Jun Sun, Boyu Yang, Jiahao Zhang, Ning Ma, Chencheng Wu, Siqing Zhang, Yiou Huang, Qiufeng Wang et al.*

    Pretrained Vision-Language-Action (VLA) policies have achieved strong single-step manipulation, but their inference remains largely memoryless, which is brittle in non-Markovian long-horizon settings with

occlusion, state aliasing, and subtle post-action changes. Prior approaches inject history either by stacking frames, which scales visual tokens and latency while adding near-duplicate pixels, or by learning additional temporal interfaces that require (re-)training and may break the original single-frame inference graph.

15. **Global Prior Meets Local Consistency: Dual-Memory Augmented Vision-Language-Action Model for Efficient Robotic Manipulation** (arXiv 2026, arXiv 2602.20200). *Zaijing Li, Bing Hu, Rui Shao, Gongwei Chen, Dongmei Jiang, Pengwei Xie, Jianye Hao, Liqiang Nie*

    Hierarchical Vision-Language-Action (VLA) models have rapidly become a dominant paradigm for robotic manipulation. It typically comprising a Vision-Language backbone for perception and understanding, together with a generative policy for action generation.

16. **Efficient Long-Horizon Vision-Language-Action Models via Static-Dynamic Disentanglement** (arXiv 2026, arXiv 2602.03983). *Weikang Qiu, Huashuo Lei, Tinglin Huang, Rex Ying*

    Vision-Language-Action (VLA) models have recently emerged as a promising paradigm for generalist robotic control. Built upon vision-language model (VLM) architectures, VLAs predict actions conditioned on visual observations and language instructions, achieving strong performance and generalization across tasks.

17. **LoLA: Long Horizon Latent Action Learning for General Robot Manipulation** (arXiv 2025, arXiv 2512.20166). *Xiaofan Wang, Xingyu Gao, Jianlong Fu, Zuolei Li, Dean Fortier, Galen Mullins, Andrey Kolobov, Baining Guo*

    The capability of performing long-horizon, language-guided robotic manipulation tasks critically relies on leveraging historical information and generating coherent action sequences. However, such capabilities are often overlooked by existing Vision-Language-Action (VLA) models.

18. **HiF-VLA: Hindsight, Insight and Foresight through Motion Representation for Vision-Language-Action Models** (arXiv 2025, arXiv 2512.09928). *Minghui Lin, Pengxiang Ding, Shu Wang, Zifeng Zhuang, Yang Liu, Xinyang Tong, Wenxuan Song, Shangke Lyu et al.*

    Vision-Language-Action (VLA) models have recently enabled robotic manipulation by grounding visual and linguistic cues into actions. However, most VLAs assume the Markov property, relying only on the current observation and thus suffering from temporal myopia that degrades long-horizon coherence.

19. **CycleManip: Enabling Cyclic Task Manipulation via Effective Historical Perception and Understanding** (arXiv 2025, arXiv 2512.01022). *Yi-Lin Wei, Haoran Liao, Yuhao Lin, Pengyue Wang, Zhizhao Liang, Guiliang Liu, Wei-Shi Zheng*

    In this paper, we explore an important yet underexplored task in robot manipulation: cycle-based manipulation, where robots need to perform cyclic or repetitive actions with an expected terminal time. These tasks are crucial in daily life, such as shaking a bottle or knocking a nail.

20. **AVA-VLA: Improving Vision-Language-Action models with Active Visual Attention** (arXiv 2025, arXiv 2511.18960). *Lei Xiao, Jifeng Li, Juntao Gao, Feiyang Ye, Yan Jin, Jingjing Qian, Jing Zhang, Yong Wu et*

al.

Vision-Language-Action (VLA) models have shown remarkable progress in embodied tasks recently, but most methods process visual observations independently at each timestep. This history-agnostic design treats robot manipulation as a Markov Decision Process, even though real-world robotic control is inherently partially observable and requires reasoning over past interactions.

21. **ContextVLA: Vision-Language-Action Model with Amortized Multi-Frame Context** (arXiv 2025, arXiv 2510.04246). *Huiwon Jang, Sihyun Yu, Heeseung Kwon, Hojin Jeon, Younggyo Seo, Jinwoo Shin*

    Leveraging temporal context is crucial for success in partially observable robotic tasks. However, prior work in behavior cloning has demonstrated inconsistent performance gains when using multi-frame observations.

22. **HAMLET: Switch your Vision-Language-Action Model into a History-Aware Policy** (arXiv 2025, arXiv 2510.00695). *Myungkyu Koo, Daewon Choi, Taeyoung Kim, Kyungmin Lee, Changyeon Kim, Younggyo Seo, Jinwoo Shin*

    Inherently, robotic manipulation tasks are history-dependent: leveraging past context could be beneficial. However, most existing Vision-Language-Action models (VLAs) have been designed without considering this aspect, i.e., they rely solely on the current observation, ignoring preceding context.

23. **MemoryVLA: Perceptual-Cognitive Memory in Vision-Language-Action Models for Robotic Manipulation** (arXiv 2025, arXiv 2508.19236). *Hao Shi, Bin Xie, Yingfei Liu, Lin Sun, Fengrong Liu, Tiancai Wang, Erjin Zhou, Haoqiang Fan et al.*

    Temporal context is essential for robotic manipulation because such tasks are inherently non-Markovian, yet mainstream VLA models typically overlook it and struggle with long-horizon, temporally dependent tasks. Cognitive science suggests that humans rely on working memory to buffer short-lived representations for immediate control, while the hippocampal system preserves verbatim episodic details and semantic gist of past experience for long-term memory.

24. **CronusVLA: Towards Efficient and Robust Manipulation via Multi-Frame Vision-Language-Action Modeling** (arXiv 2025, arXiv 2506.19816). *Hao Li, Shuai Yang, Yilun Chen, Xinyi Chen, Xiaoda Yang, Yang Tian, Hanqing Wang, Tai Wang et al.*

    Recent vision-language-action (VLA) models built on pretrained vision-language models (VLMs) have demonstrated strong performance in robotic manipulation. However, these models remain constrained by the single-frame image paradigm and fail to fully leverage the temporal information offered by multi-frame histories, as directly feeding multiple frames into VLM backbones incurs substantial computational overhead and inference latency.

25. **TraceVLA: Visual Trace Prompting Enhances Spatial-Temporal Awareness for Generalist Robotic Policies** (ICLR 2025, arXiv 2412.10345). *Ruijie Zheng, Yongyuan Liang, Shuaiyi Huang, Jianfeng Gao, Hal Daumé, Andrey Kolobov, Furong Huang, Jianwei Yang*

Although large vision-language-action (VLA) models pretrained on extensive robot datasets offer promising generalist policies for robotic learning, they still struggle with spatial-temporal dynamics in interactive robotics, making them less effective in handling complex tasks, such as manipulation. In this work, we introduce visual trace prompting, a simple yet effective approach to facilitate VLA models' spatial-temporal awareness for action prediction by encoding state-action trajectories visually.

Latent, Recurrent and Slot Memory

History compressed into fixed-size vectors or slots, updated recursively. TRACE addresses slots with trajectory signatures; Chronos, TFP and RB-VLA argue history should be the policy's latent state; muVLA isolates the capability envelope of minimal recurrence; GMP and PTP handle the spurious correlations that long histories introduce.

1. **FM-VLA: Force-based Memory for Vision-Language-Action Models in Contact-Rich Manipulation** (arXiv 2026, arXiv 2607.18231). *Ruicheng Li, Qixiu Li, Ruichun Ma, Yu Deng, Lin Luo, Zhiying Du, Jianfeng Xiang, Huizhi Liang et al.*

   Vision-language-action (VLA) models have achieved impressive generalization in robotic manipulation, and recent memory-augmented VLAs have relaxed the Markovian assumption by conditioning on past images or language summaries. Vision-based memory approaches address this by conditioning on sampled past image frames, but they are computationally expensive and fundamentally limited when temporal events are visually ambiguous, e.g., pushing a button multiple times with small movements.

2. **TFP: Temporally Conditioned Memory-Fusion Policies for Visuomotor Learning** (arXiv 2026, arXiv 2607.08283). *Yushen Liang, Yue Peng, Baosheng Jin, Tianluo Zhang, Xinyu Zhang, Shuyi Zhou, Zhuoran Chen, Xinqi Liu et al.*

   Vision–Language–Action (VLA) policies such as MATH0 and OpenVLA perform well on many manipulation tasks, but they are often reactive: the next action is predicted from the current observation, instruction, and proprioceptive state. This assumption breaks down in stage-dependent manipulation, where visually similar states may require different actions depending on latent task progress and previous interaction outcomes.

3. **ChronoFlow-Policy: Unifying Past-Current-Future Interaction Flow in Visuomotor Policy Learning** (arXiv 2026, arXiv 2606.31493). *Bokai Lin, Yifu Xu, Xinyu Zhan, Hongjie Fang, Jialin Tian, Fu-Cheng Zhang, Yong-Lu Li, Cewu Lu et al.*

   Visual signals play a crucial role in policy learning by enabling models to capture object motion and interaction dynamics. Just as humans reason about actions using both past experience and anticipated outcomes, effective policies should integrate past interactions with future predictions.

4. **Chronos: A Physics-Informed Full-History Framework for Non-Markovian Long-Horizon Manipulation** (arXiv 2026, arXiv 2606.30318). *Yulin Zhou, Yimeng Wang, Nengyu Wang, Shaojia Xing, Shiyun Tu, Xiang Li, Jingkai Zhang, Ningbo Jiang et al.*

General-purpose robot policies should be modeled as dynamical systems, yet many VLA and generative imitation policies still rely on present observations or short windows. This Markovian shortcut fails in memory-dependent manipulation: identical observations can demand different actions after different histories.

5. **Remember what you did?: Learning Behavioral Memories for Partially Observable Object Manipulation** (arXiv 2026, arXiv 2606.21188). *Kuancheng Wang, Seungho Yeom, Jinglin Cao, Yuheng Zhi, Nikhil Shinde, Michael Yip*

   Long horizon, contact-rich manipulation is inherently partially observable. This is as a single visual observation rarely captures a robot's full action context, including prior attempts, interactions, or progress.

6. **MATH1VLA: On Recurrent Memory for Partially Observable Manipulation in VLA Models** (arXiv 2026, arXiv 2606.12497). *Egor Cherepanov, Nikita Kachaev, Daniil Zelezetsky, Aydar Bulatov, Artem Pshenitsyn, Yuri Kuratov, Alexey Skrynnik, Aleksandr I. Panov et al.*

   Vision-language-action (VLA) models predict chunks of future actions from the current observation, an assumption that fails under partial observability, where decisions depend on information no longer visible. Existing memory-augmented VLAs simultaneously introduce recurrence, retrieval, compression modules, auxiliary objectives, hierarchical memory, or task-specific architectural changes, so the contribution of recurrence itself remains entangled with surrounding machinery.

7. **Action-Effect Memory Pretraining for Robot Manipulation** (arXiv 2026, arXiv 2606.12499). *Yijing Zhou, Qiwei Liang, Sitong Zhuang, Jiaxi Li, Xianpeng Wang, Boyang Cai, Yunyang Mo, Renjing Xu*

   We present AEM, an Action-Effect Memory pretraining framework for robot manipulation that learns compact temporal representations from vision-action history. Unlike prior robot representation pretraining methods that mainly focus on single-frame visual encoding, AEM targets the temporal nature of manipulation, where the current observation alone is often insufficient under partial observability.

8. **DSSP: Diffusion State Space Policy with Full-History Encoding** (arXiv 2026, arXiv 2605.14598). *Zhiyuan Guan, Jianshu Hu, Han Fang, Yunpeng Jiang, Yize Huang, Shujia Li, Xiao Li, Yutong Ban*

   Diffusion-based imitation learning has shown strong promise for robot manipulation. However, most existing policies condition only on the current observation or a short window of recent observations, limiting their ability to resolve history-dependent ambiguities in long-horizon tasks.

9. **Gated Memory Policy: In-Context Memorization and Adaptation** (arXiv 2026, arXiv 2604.18933). *Yihuai Gao, Jeff Jinyun Liu, Shuang Li, Shuran Song*

   Robotic manipulation tasks exhibit varying memory requirements, ranging from Markovian tasks that require no memory to non-Markovian tasks that demand in-context memorization of historical information within a single trial or in-context adaptation based on the outcomes of multiple past trials. Surprisingly, simply extending observation histories of a visuomotor policy often leads to a significant performance drop due to distribution shift and overfitting.

10. **MemoAct: Atkinson-Shiffrin-Inspired Hierarchical Memory-Augmented Policy for Robotic Manipulation** (arXiv 2026, arXiv 2603.18494). *Liufan Tan, Jiale Li, Gangshan Jing*

    Memory-augmented robotic policies are essential in handling memory-dependent tasks. However, existing approaches typically rely on simply extending the observation window, struggling to simultaneously achieve precise task-state tracking and robust long-horizon retention.

11. **Beyond Short-Horizon: VQ-Memory for Robust Long-Horizon Manipulation in Non-Markovian Simulation Benchmarks** (arXiv 2026, arXiv 2603.09513). *Honghui Wang, Zhi Jing, Jicong Ao, Shiji Song, Xuelong Li, Gao Huang, Chenjia Bai*

    The high cost of collecting real-robot data has made robotic simulation a scalable platform for both evaluation and data generation. Yet most existing benchmarks concentrate on simple manipulation tasks such as pick-and-place, failing to capture the non-Markovian characteristics of real-world tasks and the complexity of articulated object interactions.

12. **VPWEM: Non-Markovian Visuomotor Policy with Working and Episodic Memory** (arXiv 2026, arXiv 2603.04910). *Yuheng Lei, Zhixuan Liang, Hongyuan Zhang, Ping Luo*

    Imitation learning from human demonstrations has achieved significant success in robotic control, yet most visuomotor policies still condition on single-step observations or short-context histories, making them struggle with non-Markovian tasks that require long-term memory. Simply enlarging the context window incurs substantial computational and memory costs and encourages overfitting to spurious correlations, leading to catastrophic failures under distribution shift and violating real-time constraints in robotic systems.

13. **Recursive Belief Vision Language Action Models** (arXiv 2026, arXiv 2602.20659). *Vaidehi Bagaria, Bijo Sebastian, Nirav Kumar Patel*

    Vision-language-action models must enable agents to execute long-horizon tasks under partial observability. However, most existing approaches remain observation-driven, relying on short context windows or repeated queries to vision-language models (VLMs).

14. **Rethinking Progression of Memory State in Robotic Manipulation: An Object-Centric Perspective** (arXiv 2025, arXiv 2511.11478). *Nhat Chung, Taisei Hanyu, Toan Nguyen, Huy Le, Frederick Bumgarner, Duy Minh Ho Nguyen, Khoa Vo, Kashu Yamazaki et al.*

    As embodied agents operate in increasingly complex environments, the ability to perceive, track, and reason about individual object instances over time becomes essential, especially in tasks requiring sequenced interactions with visually similar objects. In these non-Markovian settings, key decision cues are often hidden in object-specific histories rather than the current scene.

15. **ELMUR: External Layer Memory with Update/Rewrite for Long-Horizon RL Problems** (arXiv 2025, arXiv 2510.07151). *Egor Cherepanov, Alexey K. Kovalev, Aleksandr I. Panov*

    Real-world robotic agents must act under partial observability and long horizons, where key cues may appear long before they affect decision making. However, most modern approaches rely solely on

instantaneous information, without incorporating insights from the past.

16. **MEMBOT: Memory-Based Robot in Intermittent POMDP** (arXiv 2025, arXiv 2509.11225). *Youzhi Liang, Eyan Noronha*

    Robotic systems deployed in real-world environments often operate under conditions of partial and often intermittent observability, where sensor inputs may be noisy, occluded, or entirely unavailable due to failures or environmental constraints. Traditional reinforcement learning (RL) approaches that assume full state observability are ill-equipped for such challenges.

17. **MTIL: Encoding Full History with Mamba for Temporal Imitation Learning** (arXiv 2025, arXiv 2505.12410). *Yulin Zhou, Yuankai Lin, Fanzhe Peng, Jiahui Chen, Kaiji Huang, Hua Yang, Zhouping Yin*

    Standard imitation learning (IL) methods have achieved considerable success in robotics, yet often rely on the Markov assumption, which falters in long-horizon tasks where history is crucial for resolving perceptual ambiguity. This limitation stems not only from a conceptual gap but also from a fundamental computational barrier: prevailing architectures like Transformers are often constrained by quadratic complexity, rendering the processing of long, high-dimensional observation sequences infeasible.

18. **Learning Long-Context Diffusion Policies via Past-Token Prediction** (arXiv 2025, arXiv 2505.09561). *Marcel Torne, Andy Tang, Yuejiang Liu, Chelsea Finn*

    Reasoning over long sequences of observations and actions is essential for many robotic tasks. Yet, learning effective long-context policies from demonstrations remains challenging.

19. **SAM2Act: Integrating Visual Foundation Model with A Memory Architecture for Robotic Manipulation** (arXiv 2025, arXiv 2501.18564). *Haoquan Fang, Markus Grotz, Wilbert Pumacay, Yi Ru Wang, Dieter Fox, Ranjay Krishna, Jiafei Duan*

    Robotic manipulation systems operating in diverse, dynamic environments must exhibit three critical abilities: multitask interaction, generalization to unseen scenarios, and spatial memory. While significant progress has been made in robotic manipulation, existing approaches often fall short in generalization to complex environmental variations and addressing memory-dependent tasks.

20. **Learning Memory Mechanisms for Decision Making through Demonstrations** (arXiv 2024, arXiv 2411.07954). *William Yue, Bo Liu, Peter Stone*

    In Partially Observable Markov Decision Processes, integrating an agent's history into memory poses a significant challenge for decision-making. Traditional imitation learning, relying on observation-action pairs for expert demonstrations, fails to capture the expert's memory mechanisms used in decision-making.

Dual-System, Agentic and Symbolic Memory

A VLM / LLM / code layer maintains text, graphs or progress pointers; a low-level VLA executes. The leaders on RoboMME and RoboMemArena (PonderPounce, HyMeS, BATON) live here: for counting and procedural memory, symbolic state is currently steadier. AGM's thesis that reliable memory comes from

disciplined state updates rather than capacity is the same principle as EventVLA's NMS plus cooldown and TRACE's gated writes.

1. **AGM: Achievement-Grounded Memory for Closed-Loop Agents with Frozen VLA Policies** (arXiv 2026, arXiv 2608.29537). *Hongbo Gao, Zeyu Ni, Xin Wen, Siyu Xu, Ruifeng Li*

   Frozen vision-language-action (VLA) policies offer broad manipulation skills but execute open-loop action chunks without tracking task progress, so the agent cannot reliably decide whether to continue, retry, or terminate. External memory is a natural remedy, yet it can be harmful when attempted actions are treated as completed progress, turning local execution errors into persistent task-state errors.

2. **PonderPounce: A Pretrained MLLM as an Episode Context Engine for Robot Control** (arXiv 2026, arXiv 2608.24115). *Suhwan Choi, Jaeyoon Jung, Sungkyung Kim, Yunsung Lee, Youngjae Yu*

   Multimodal large language models (MLLMs) can integrate long visual histories, reason under partial observability, and infer behavior from a few examples. Yet vision-language-action (VLA) models generally inherit pretrained representations without using this contextual capacity as episode memory.

3. **Don't Drop the BATON: Long-Horizon Robot Manipulation via Agentic Subtask Exploration and Transition-aware Memory** (arXiv 2026, arXiv 2608.16889). *Bingxin Xu, Yuzhang Shang, Emilio Ferrara*

   Long-horizon robot manipulation chains many contact-rich skills into one multi-stage task. Vision-language-action (VLA) models increasingly master the individual skills, yet the chain still fails: errors compound beyond the policy's ability to correct, and one subtask silently constrains the next.

4. **Skills in Weights, Memory in Code: Hybrid Learning for Memory-Dependent Robot Manipulation** (arXiv 2026, arXiv 2608.09410). *Yunhao Zhao, Zhenyang Ni, Haoyang Chen, Ruohan Zhang, Qi Zhu*

   Modern vision-language-action (VLA) policies have acquired broad manipulation skills, but typically generate each action chunk from the current observation or a short fixed-length history. However, real-world manipulation is often non-Markovian, requiring robots to retain and reason over task-relevant information from long-horizon interaction histories to determine the next action.

5. **OnEvoMemory: Evolving Memory through Online Robot Rollouts for Pretrained Robot Policies** (arXiv 2026, arXiv 2608.08749). *Zhongxi Chen, Shenqi Zong*

   Long-horizon robot manipulation requires policies to track completed subtasks and critical interaction events. However, existing memory mechanisms heavily rely on external models or predefined update rules.

6. **SkillMemo: Expert-guided Skill Memory Framework for Compositional Embodied Manipulation** (arXiv 2026, arXiv 2608.05970). *Changyuan Wang, Chubin Zhang, Zhenyu Wu, Runhao Li, Angyuan Ma, Ke Chao, Yinan Liang, Xiuwei Xu et al.*

   Embodied visuomotor models, including Diffusion Policy (DP) and Vision-Language-Action (VLA) models, have demonstrated promising performance on robotic manipulation benchmarks. However, their potential remains fundamentally constrained by the scarcity of large-scale embodied trajectory

datasets, leading to insufficient compositional generalization in out-of-distribution (OOD) scenarios with limited capability to capture reusable skill structures.

7. **Explicit Language Memory for Long-Horizon Planning in Vision-Language-Action Models** (arXiv 2026, arXiv 2608.04765). *Houze Xu, Jizhong Li, Ziyi Ye*

   Vision-language-action (VLA) models provide a unified paradigm for connecting visual perception, language understanding, and robotic control. However, existing VLA models still face major challenges in long-horizon tasks: sparse expert demonstrations constrain cross-task compositional generalization; the non-Markovian nature of long-horizon tasks makes it difficult for policies conditioned only on current observations to maintain temporal consistency; limited closed-loop error correction allows execution errors to accumulate; and end-to-end action fine-tuning may weaken the high-level semantic representations of vision-language model (VLM) backbones.

8. **ChainVLA: Chaining Vision-Language-Action Queries through a Unified Execution State for Long-Horizon Manipulation** (arXiv 2026, arXiv 2608.02326). *Yuzhi Huang, Weijue Bu, Ziyi Xiong, Jie Wu, Fanding Huang, Jingyan Jiang, Zhi Wang*

   Humans perform long-horizon manipulation by retaining knowledge of what earlier actions have established while continuously adapting the motion underway. By contrast, action-chunked vision-language-action (VLA) policies repeatedly replan from the current input at each query.

9. **Harness VLA: Steering Frozen VLAs into Reliable Manipulation Primitives via Memory-Guided Agents** (arXiv 2026, arXiv 2607.08448). *Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Chunyang Zhu, Jiaxing Qiu, Yuchen Yan et al.*

   Language-conditioned manipulation requires both precise contact-rich control and robust reasoning over language, scenes, and long horizons. End-to-end Vision-Language-Action (VLA) models provide strong local visuomotor skills, but they are trained on in-distribution task trajectories and often fail under deployment perturbations such as semantic retargeting, goal re-binding, spatial-layout shifts, and unstable local contacts.

10. **GeneralVLA-2: Geometry-Aware Reconstruction and Governed Memory for Robot Planning** (arXiv 2026, arXiv 2606.17480). *Haoyu Wang, Guoqing Ma, Zeyu Zhang, Yandong Guo, Boxin Shi, Hao Tang*

    Generalist vision-language-action systems need object-centric 3D evidence and reusable manipulation experience to plan reliable robot trajectories. GeneralVLA provides a hierarchical interface for converting language and RGB-D observations into 3D end-effector paths, but two bottlenecks remain.

11. **CodeGraphVLP: Code-as-Planner Meets Semantic-Graph State for Non-Markovian Vision-Language-Action Models** (arXiv 2026, arXiv 2604.22238). *Khoa Vo, Sieu Tran, Taisei Hanyu, Yuki Ikebe, Duy Nguyen, Nghi D. Q. Bui, Minh Vu, Anthony Gunderman et al.*

    Vision-Language-Action (VLA) models promise generalist robot manipulation, but are typically trained and deployed as short-horizon policies that assume the latest observation is sufficient for action reasoning. This assumption breaks in non-Markovian long-horizon tasks, where task-relevant evidence

can be occluded or appear only earlier in the trajectory, and where clutter and distractors make fine-grained visual grounding brittle.

12. **HELM: Harness-Enhanced Long-horizon Memory for Vision-Language-Action Manipulation** (arXiv 2026, arXiv 2604.18791). *Zijian Zeng, Fei Ding, Huiming Yang, Xianwei Li*

    Vision-Language-Action (VLA) models fail systematically on long-horizon manipulation tasks despite strong short-horizon performance. We show that this failure is not resolved by extending context length alone in the current reactive execution setting; instead, it stems from three recurring execution-loop deficiencies: the memory gap, the verification gap, and the recovery gap.

13. **MEM: Multi-Scale Embodied Memory for Vision Language Action Models** (arXiv 2026, arXiv 2603.03596). *Marcel Torne, Karl Pertsch, Homer Walke, Kyle Vedder, Suraj Nair, Brian Ichter, Allen Z. Ren, Haohuan Wang et al.*

    Conventionally, memory in end-to-end robotic learning involves inputting a sequence of past observations into the learned policy. However, in complex multi-stage real-world tasks, the robot's memory must represent past events at multiple levels of granularity: from long-term memory that captures abstracted semantic concepts (e.g., a robot cooking dinner should remember which stages of the recipe are already done) to short-term memory that captures recent events and compensates for occlusions (e.g., a robot remembering the object it wants to pick up once its arm occludes it).

14. **Notes-to-Self: Scratchpad Augmented VLAs for Memory Dependent Manipulation Tasks** (arXiv 2026, arXiv 2602.21013). *Sanjay Haresh, Daniel Dijkman, Apratim Bhattacharyya, Roland Memisevic*

    Many dexterous manipulation tasks are non-markovian in nature, yet little attention has been paid to this fact in the recent upsurge of the vision-language-action (VLA) paradigm. Although they are successful in bringing internet-scale semantic understanding to robotics, existing VLAs are primarily "stateless" and struggle with memory-dependent long horizon tasks.

15. **Action-Sketcher: From Reasoning to Action via Visual Sketches for Long-Horizon Robotic Manipulation** (arXiv 2026, arXiv 2601.01618). *Huajie Tan, Peterson Co, Yijie Xu, Shanyu Rong, Yuheng Ji, Cheng Chi, Xiansheng Chen, Qiongyu Zhang et al.*

    Long-horizon robotic manipulation is increasingly important for real-world deployment, requiring spatial disambiguation in complex layouts and temporal resilience under dynamic interaction. However, existing end-to-end and hierarchical Vision-Language-Action (VLA) policies often rely on text-only cues while keeping plan intent latent, which undermines referential grounding in cluttered or underspecified scenes, impedes effective task decomposition of long-horizon goals with close-loop interaction, and limits causal explanation by obscuring the rationale behind action choices.

16. **EchoVLA: Robotic Vision-Language-Action Model with Synergistic Declarative Memory for Mobile Manipulation** (arXiv 2025, arXiv 2511.18112). *Min Lin, Xiwen Liang, Bingqian Lin, Jingzhi Liu, Zijian Jiao, Kehan Li, Ziang Yan, Yu Sun et al.*

    Recent progress in Vision-Language-Action (VLA) models has enabled embodied agents to interpret multimodal instructions and perform complex tasks. However, existing VLAs are mostly confined to

short-horizon, table-top manipulation, lacking the memory and reasoning capability required for mobile manipulation, where agents must coordinate navigation and manipulation under changing spatial contexts.

17. **MAP-VLA: Memory-Augmented Prompting for Vision-Language-Action Model in Robotic Manipulation** (arXiv 2025, arXiv 2511.09516). *Runhao Li, Wenkai Guo, Zhenyu Wu, Changyuan Wang, Haoyuan Deng, Zhenyu Weng, Yap-Peng Tan, Ziwei Wang*

    Pre-trained Vision-Language-Action (VLA) models have achieved remarkable success in improving robustness and generalization for end-to-end robotic manipulation. However, these models struggle with long-horizon tasks due to their lack of memory and reliance solely on immediate sensory inputs.

18. **ExpReS-VLA: Specializing Vision-Language-Action Models Through Experience Replay and Retrieval** (arXiv 2025, arXiv 2511.06202). *Shahram Najam Syed, Yatharth Ahuja, Arthur Jakobsson, Jeff Ichnowski*

    Vision-Language-Action (VLA) models like OpenVLA demonstrate impressive zero-shot generalization across robotic manipulation tasks but struggle to adapt to specific deployment environments where consistent high performance on a limited set of tasks is more valuable than broad generalization. We present EXPierence replayed, REtrieval augmented, Specialized VLA (ExpReS-VLA), a method that enables rapid on-device adaptation of pre-trained VLAs to target domains while preventing catastrophic forgetting through compressed experience replay and retrieval-augmented generation.

19. **MemER: Scaling Up Memory for Robot Control via Experience Retrieval** (arXiv 2025, arXiv 2510.20328). *Ajay Sridhar, Jennifer Pan, Satvik Sharma, Chelsea Finn*

    Humans routinely rely on memory to perform tasks, yet most robot policies lack this capability; our goal is to endow robot policies with the same ability. Naively conditioning on long observation histories is computationally expensive and brittle under covariate shift, while indiscriminate subsampling of history leads to irrelevant or redundant information.

Memory inside World and Video Action Models

Memory serves prediction of the future, and prediction serves action. MemoryWAM's recent frames + event-boundary anchors + gist tokens is structurally the same as EventVLA's anchors plus event frames.

1. **MemoryWAM: Efficient World Action Modeling with Persistent Memory** (arXiv 2026, arXiv 2606.20562). *Sizhe Yang, Juncheng Mu, Tianming Wei, Chenhao Lu, Xiaofan Li, Linning Xu, Zhengrong Xue, Zhecheng Yuan et al.*

    Robust robotic manipulation in the real world requires not only an understanding of the current observation, but also memory and dynamics modeling. World action models (WAMs) possess these capabilities by jointly modeling visual foresight and actions conditioned on both current and historical observations, making them a promising paradigm for robotic manipulation.

2. **MemoryVAM: Integrating Memory into Video Action Model for Robot Manipulation** (arXiv 2026, arXiv 2606.20679). *Yuxin Jiang, Chang Yu, Yunuo Chen, Xiang Feng, Yin Yang, Nishank Gite, Chenfanfu*

Jiang

Video-world-model policies learn action-relevant representations by predicting future observations. However, they condition on only a short observation window, which renders long-horizon manipulation non-Markovian when the correct action depends on earlier events that are no longer visible.

3. **TriVLA: A Triple-System-Based Unified Vision-Language-Action Model with Episodic World Modeling for General Robot Control** (arXiv 2025, arXiv 2507.01424). *Zhenyang Liu, Yongchong Gu, Sixiao Zheng, Yanwei Fu, Xiangyang Xue, Yu-Gang Jiang*

   Recent advances in vision-language models (VLMs) have enabled robots to follow open-ended instructions and demonstrate impressive commonsense reasoning. However, current vision-language-action (VLA) frameworks primarily rely on static representations and limited temporal context, restricting agents to short-horizon, reactive behaviors and hindering robust generalization in dynamic embodied environments.

Multi-Robot Collaboration and Partner Memory

A partner's phase is a latent variable that can only be inferred from history, so multi-robot collaboration is the same non-Markovian problem in another guise. Current work is almost entirely about data collection (HATS, Duet, Tri-Manual); nobody has yet used an explicit memory module for partner-state estimation, a clear gap.

1. **Making two action heads agree: coordination mechanisms and a runtime collapse certificate for flow-matching policies** (arXiv 2026, arXiv 2608.15748). *Jinhui Sun, Wei Zhou, Bowen Yang, Xinliang Xiao, Li Yang*

   A dual-representation flow-matching policy decodes each predicted motion into joint and end-effector spaces, and the residual between the two kinematically equivalent decodings provides a physically interpretable runtime signal. On multimodal tasks, however, independently sampled branches may choose different valid modes, causing false alarms.

2. **AutoIntervene: Calibrated Intervention for Action-Chunking Imitation Learning Policies** (arXiv 2026, arXiv 2608.07065). *Jinhe Tang, Weiming Zhi*

   Action-chunking visuomotor policies learn from demonstrations and improve temporal consistency by predicting short action sequences rather than single-step commands. Yet perception errors and execution drift can move the robot outside the demonstration distribution, while the policy continues to produce smooth action chunks that are inconsistent with the observed state.

3. **Tri-Manual Visuomotor Imitation Learning of Robot Policies** (arXiv 2026, arXiv 2607.25731). *James Zhao, Mingyuan Ba, Weiming Zhi*

   Bimanual teleoperation provides an effective way to collect robot demonstrations, but it assumes that the operator and robot have matching numbers of simultaneous control channels. This assumption breaks for tri-manual systems: the robot can coordinate three arms concurrently, whereas a single operator can continuously control only two.

4. **Duet: Dual-Robot Understanding via Efficient Teaching** (arXiv 2026, arXiv 2606.20990). *Yiqi Zhao, Ruohai Ge, Celina Shiyu Wang, Junjie Ye, Muchen Xu, Minhao Li, Sergey Zakharov, Basile Van Hoorick et al.*

   Dual-robot collaboration enables tasks that exceed the reach and payload of a single robot, such as collaboratively transporting objects across environments and executing coordinated handovers. Data acquisition is the primary bottleneck for training these systems.

5. **HATS: A Human-Agent Teleoperation System for Multi-Arm Data Collection** (arXiv 2026, arXiv 2606.16491). *Zesen Lin, Jian-Jian Jiang, Haoming Cen, Xiao-Ming Wu, Dandan Zhang, Wei-Shi Zheng*

   Many real-world manipulation scenarios, such as handling complex collaborative tasks and dealing with large workspaces, require coordination of more than two robotic arms. Consequently, an effective multi-arm teleoperation system is required to collect demonstrations for training coordinated multi-arm manipulation policies.

6. **TriPilot-FF: Coordinated Whole-Body Teleoperation with Force Feedback** (arXiv 2026, arXiv 2602.09888). *Zihao Li, Yanan Zhou, Ranpeng Qiu, Hangyu Wu, Guoqiang Ren, Weiming Zhi*

   Mobile manipulators broaden the operational envelope for robot manipulation. However, the whole-body teleoperation of such robots remains a problem: operators must coordinate a wheeled base and two arms while reasoning about obstacles and contact.

7. **Rethinking Bimanual Robotic Manipulation: Learning with Decoupled Interaction Framework** (ICCV 2025, arXiv 2503.09186). *Jian-Jian Jiang, Xiao-Ming Wu, Yi-Xiang He, Ling-An Zeng, Yi-Lin Wei, Dandan Zhang, Wei-Shi Zheng*

   Bimanual robotic manipulation is an emerging and critical topic in the robotics community. Previous works primarily rely on integrated control models that take the perceptions and states of both arms as inputs to directly predict their actions.

8. **IntervenGen: Interventional Data Generation for Robust and Data-Efficient Robot Imitation Learning** (IROS 2024, arXiv 2405.01472). *Ryan Hoque, Ajay Mandlekar, Caelan Garrett, Ken Goldberg, Dieter Fox*

   Imitation learning is a promising paradigm for training robot control policies, but these policies can suffer from distribution shift, where the conditions at evaluation time differ from those in the training data. A popular approach for increasing policy robustness to distribution shift is interactive imitation learning (i.e., DAgger and variants), where a human operator provides corrective interventions during policy rollouts.

9. **Mobile ALOHA: Learning Bimanual Mobile Manipulation with Low-Cost Whole-Body Teleoperation** (CoRL 2024, arXiv 2401.02117). *Zipeng Fu, Tony Z. Zhao, Chelsea Finn*

   Imitation learning from human demonstrations has shown impressive performance in robotics. However, most results focus on table-top manipulation, lacking the mobility and dexterity necessary for generally useful tasks.

Background: Memory Mechanisms and Trajectory Descriptors

The minimum background for the three core papers: the origins of external memory and memory tokens (MANN, RMT, Titans), path signatures (Signatory, CILO) and interactive imitation (DAgger).

1. **Titans: Learning to Memorize at Test Time** (arXiv 2025, arXiv 2501.00663). *Ali Behrouz, Peilin Zhong, Vahab Mirrokni*

   Over more than a decade there has been an extensive research effort on how to effectively utilize recurrent models and attention. While recurrent models aim to compress the data into a fixed-size memory (called hidden state), attention allows attending to the entire context window, capturing the direct dependencies of all tokens.

2. **Explorative Imitation Learning: A Path Signature Approach for Continuous Environments** (arXiv 2024, arXiv 2407.04856). *Nathan Gavenski, Juarez Monteiro, Felipe Meneguzzi, Michael Luck, Odinaldo Rodrigues*

   Some imitation learning methods combine behavioural cloning with self-supervision to infer actions from state pairs. However, most rely on a large number of expert trajectories to increase generalisation and human intervention to capture key aspects of the problem, such as domain constraints.

3. **Recurrent Memory Transformer** (NeurIPS 2022, arXiv 2207.06881). *Aydar Bulatov, Yuri Kuratov, Mikhail S. Burtsev*

   Transformer-based models show their effectiveness across multiple domains and tasks. The self-attention allows to combine information from all sequence elements into context-aware representations.

4. **Signatory: differentiable computations of the signature and logsignature transforms, on both CPU and GPU** (ICLR 2021, arXiv 2001.00706). *Patrick Kidger, Terry Lyons*

   Signatory is a library for calculating and performing functionality related to the signature and logsignature transforms. The focus is on machine learning, and as such includes features such as CPU parallelism, GPU support, and backpropagation.

5. **One-shot Learning with Memory-Augmented Neural Networks** (ICML 2016, arXiv 1605.06065). *Adam Santoro, Sergey Bartunov, Matthew Botvinick, Daan Wierstra, Timothy Lillicrap*

   Despite recent breakthroughs in the applications of deep neural networks, one setting that presents a persistent challenge is that of “one-shot learning.” Traditional gradient-based networks require a lot of data to learn, often through extensive iterative training. When new data is encountered, the models must inefficiently relearn their parameters to adequately incorporate the new information without catastrophic interference.

6. **A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning** (AISTATS 2011, arXiv 1011.0686). *Stephane Ross, Geoffrey J. Gordon, J. Andrew Bagnell*

   Sequential prediction problems such as imitation learning, where future observations depend on previous predictions (actions), violate the common i.i.d. assumptions made in statistical learning.

Appendix: deliverables and reproduction

Deliverable	Path
English PDFs of the three core papers	`papers/pdf/`
Layout-preserving Chinese translations	`papers/zh/`
Deep-dive reports (zh/en, md + pdf)	`reports/` , `reports/pdf/`
Trends and insights (zh/en)	`reports/04_trends_insights_{cn,en}.md`
Design-space matrix	`insights/DESIGN_SPACE_MATRIX.md`
103 paper notes	`notes/`
Consolidated report (this file)	`report/awesome_memory_vla_report_{cn,en}.{html,pdf}`
Slides	`slides/awesome_memory_vla_deck.html` , `slides/awesome_memory_vla_deck.pdf`
BibTeX	`awesome_memory_vla.bib`

Reproduce: `python3 scripts/build_manifest.py && python3 scripts/make_notes.py && python3 scripts/make_bib.py && python3 scripts/make_readme.py && python3 scripts/build_pdfs.py`. Translation: `bash scripts/translate_core.sh <KEY>` (DeepSeek) or `bash scripts/translate_fallback_google.sh` (no key).